



An integrated differential evolution algorithm for reconfigurable manufacturing systems

Ismail M. Ali^{*}, Sumana Biswas, Hasan H. Turan, Ripon K. Chakraborty, Sondoss Elsayah, Michael J. Ryan¹

Capability Systems Centre, School of Engineering and IT, University of New South Wales, Canberra, Australia

HIGHLIGHTS

- This paper reports an integrated DE for Reconfigurable Manufacturing Systems.
- Enhanced heuristics for product platforms' formation and assembly line are proposed.
- New efficient package-based mathematical model is introduced.
- Two real-world cases are used to validate the performance of the proposed MJPN-DE.
- Results demonstrated efficiency of MJPN-DE in solving Module-based PFD problems.

ARTICLE INFO

Keywords:

Differential evolution
Reconfigurable manufacturing system
Package-based model
Product family design
Assembly line

ABSTRACT

Product platforms consider an effective strategy to empower mass customization (MC) in a reconfigurable manufacturing system (RMS) to have the capacity to reconfigure the hardware and manage resources at all organisational and functional levels. This enables quick modification of production functions in reaction to unforeseen changes in the market or regulatory requirements. Although several platform formation and assembly line techniques have emerged to satisfy the increased demand for MC and enable the manufacture of products with high variety, they also increase the system's complexity and make it less cost-efficient as variety increases. This study aims to minimize the complexity of a production system by reducing the number of separate components in different production processes. An efficient approach that integrates the Median-Joining Phylogenetic Networks (MJPN) classification tool and differential evolution (DE) in one unified framework is developed to do that. The proposed framework uses the concept of the package to group separate components that are often employed jointly to produce particular products and allows the use of the 'all-unit discount policy', which applies discounts to the purchasing costs of a specific quantity of materials/components. Finally, two case studies are applied to validate the proposed MJPN-DE in different dimensions. The experimental results show that applying the proposed package model saves 57.5% of the purchasing costs of components and 41.35% average setup costs compared to MJPN- and non-platform-based models and increases the postponement effectiveness of the obtained layout by 41.51% and 74.11% compared with two other layout strategies. In algorithmic comparisons, the results of comparing MJPN-DE with standard DE and genetic algorithm (GA) show an enhanced performance of MJPN-DE where the total production cost is reduced in the small case study by 63.34% and 38.52%, and in the large case study by 16.90% and 11.51%, respectively.

1. Introduction

Permanent link to reproducible Capsule: <https://doi.org/10.24433/CO.2193541.v1>.

In modern manufacturing, companies face an elevated level of product development, market globalization, dynamic demand from customers and technological advancements [1,2]. Producing a wide range of products can meet a wide range of customers' demands while

^{*} Corresponding author.

E-mail address: Ismail.Ali@unsw.edu.au (I.M. Ali).

¹ Capability Associates, Canberra, Australia

also increasing profit and sales [3]. However, such a variety frequently raises manufacturing costs, thereby lowering profits. Many manufacturers have been seeking to provide additional product varieties to meet the growing demand for niche sectors of the market without losing manufacturing quality [4]. These trends encourage enterprises to embrace the mass customisation (MC) concept to meet individual customer needs [5]. MC is the rationale behind creating product families to satisfy different customer requirements with acceptable costs [6]. Such a strategy's key benefit is delivering various products to consumers with the equivalent quality and prices as mass-manufactured products. Many manufacturers are embracing product development based on platforms and product family design (PFD) as active ways to enable MC to boost product variation while reducing manufacturing costs and time-to-market [5]. Product platforms are developed by grouping the components that are frequently utilized to produce most of the product variants, and simultaneously, they do not have to be shared by all of the products [7]. To acquire distinct product versions, each constructed platform is subsequently differentiated by adding (i.e., assembling) or removing (i.e., disassembling) components to or from it. Each product is distinguished by its own set of features/components that are adapted to certain customers' desires [8]. Design and development of a product family are difficult in several aspects, such as choosing business policies, reflecting numerous marketing challenges and client expectations, and design phase engineering, including reconfiguring and customizing the platform to generate the final products [9]. Assembly/disassembly (reconfiguring and customising) processes in the design phase are the core elements that influence the entire cost of the manufacturing process, as they account for 20% of total production costs and roughly 50% of total production time and labour costs [7]. Thus, reducing the complexity and cost of such processes is the key objective of this study.

In the literature, the assembly line in mass production was initially proposed by Henry Ford to lower the cost of assembly and disassembly procedures [10]. The assembly line was depicted in his work as a collection of points/stations used to complete certain assembly tasks in the production of final products. Because the decision-maker (project planner) frequently creates product platforms after determining and then customizing common components, assembly system's design, planning, and balancing are not normally done simultaneously with the formation of product platforms [11]. As a result, more advanced approaches for effectively utilizing platforms to manufacture many product versions have emerged. The assembly and disassembly modular platform (ADMP) is one technique that applies components' assembly and disassembly to form platforms. ADMP has been used in [12], and their results showed that ADMP could improve platform efficiency by producing extra products using the same platform. The principle of ADMP has been proposed to avoid employing only common components within a particular platform which reduces the number of products that can be obtained from this platform [13]. The ADMP formation is practically equivalent to the ancestry evaluation in the area of biological and living organisms, which is utilized to follow their potential evolutionary paths. One of these methods is the median-joining phylogenetic network (MJPN) [14], which has been implemented by many studies in the product family's design phase to detect the number and development of several platforms and the assembly and disassembly processes to form the platforms to produce several product variants [7,15]. According to their results, MJPN has achieved an outstanding performance compared with other techniques, encouraging us to integrate it with a powerful evolutionary algorithm to assess its performance as an integrated component.

Due to the importance of PFD, numerous exact, heuristic, and meta-heuristic approaches have been proposed to optimally form product platforms and balance various changes in assembly lines [16,17]. However, in most real-world applications, the design parameters differ in various parameter domains and have numerous constraints. Consequently, achieving optimal values is a problematic and challenging task due to the complexity of such production systems [18]. The complexity

of this task is further increased when these products are part of a product family, as determining the platform is considered a more complicated and critical additional task. Such problems can be effectively solved by robust algorithms that can search for feasible optimal solutions with minimum initial problem information [19]. So, heuristic and meta-heuristic methods seem to be more efficient and promising than the exact (optimum-seeking) methods [20]. To acquire a common platform for a module-based product family design (module-based PFD), Olivares-Benitez and Gonzalez-Velarde [21] proposed a meta-heuristic approach based on scatter search (SS) and tabu search (TS), taking into account product performance and production cost in the selection method. Recently, a genetic algorithm (GA) was designed to determine the best platform formation for a large number of products [22], where several production models, such as product platform development and hybrid manufacturing, have been combined. In most of the developed techniques, the common objectives of the line balancing (assembly line) can be classified into two categories: 1) minimization of overall idle time given the required cycle time and 2) minimization of cycle time for a static number of stations [20]. Other objectives have also been identified in [23], such as 3) maximization of profit [24] by finding the optimal number of stations and 4) the minimization of anticipated overall costs [25] by reducing the sum of incompleteness cost and labour cost. However, most of the proposed metaheuristic approaches for solving product platforms concentrate on the assembly area. Other methods, which tackle the reconfigurable manufacturers, have crucial shortcomings in the number of products and features (components) to be considered in the production system [22]. Among meta-heuristics, differential evolution (DE) has a lengthy track record of successfully addressing optimization problems with continuous and discrete variables [26]. However, it still needs to improve in several aspects, including convergence speed to the optimal solution and avoiding restricting the search to a local solution rather than finding the best solution, particularly when dealing with complex combinatorial optimization problems. However, to the best of our knowledge, no technique attempts to improve/reduce the number of assembly and disassembly processes of separate components in both platforms and customization processes. Additionally, the development of assembly layouts depends mainly on the project planner's experience and creativity for combining different manufacturing parts and often does not properly integrate other essential variables, such as product family design, platform development and setup (assembly/disassembly) process, which may initiate sub-optimal solutions [27].

Motivated by the abovementioned drawbacks, in this paper, we use an integrated DE algorithm to build a reactive strategy to design a layout for the module-based product family optimally. The proposed algorithm combines the ideas of 'product platform development' and 'customization through component assembly/disassembly' into a single unified framework to minimize the total production costs, including material purchasing and customization expenses. Several elements have also been incorporated in the proposed MJPN-DE, such as the MJPN classification tool for producing the initial PFD, an improved representation of the solution to include the various segments of the PFD solution, a newly proposed package-based model to enhance the initial solutions and an enhanced best-matched value (BMV) mapping method to deal with the solution's continuous variables obtained from the DE's evolving operators. Lastly, the proposed MJPN-DE algorithm is validated by solving 1) a small-scale real-world case study of a kettle family in which 5 different kettle variants are produced using 15 components and 2) a large-sized simulated case study where 100 components are employed to produce 30 different products.

This paper is organized as follows: Section 2 reviews the literature on the module-based PFD and the assembly line of components. Section 3 provides the problem description and the mathematical formulation of the proposed package-based model. The design of the proposed integrated MJPN-DE framework is described in Section 4. Section 5 shows the computational complexity of the module-based PFD, the two case

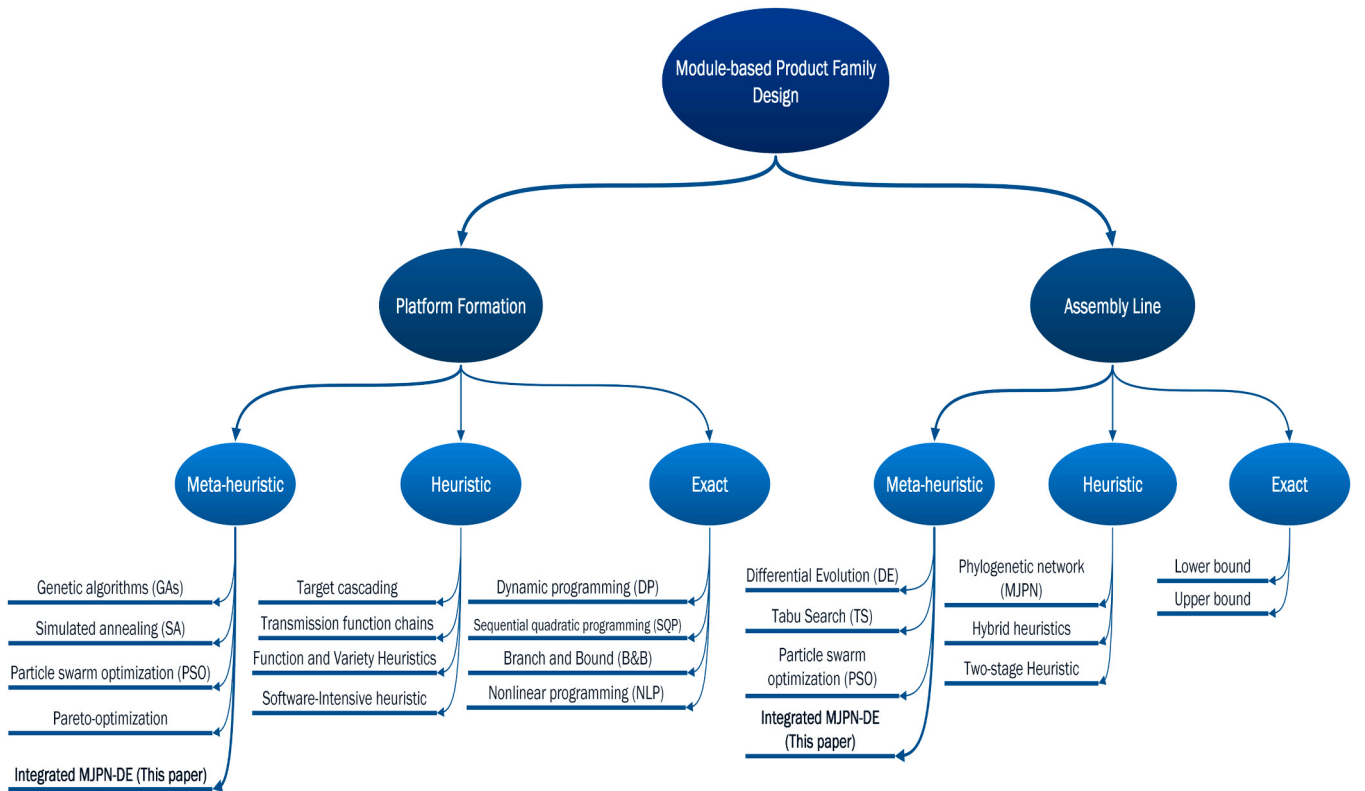


Fig. 1. Taxonomy of product platform formation and assembly line optimization techniques.

studies, the experimental results, including parameter tuning and numerical results, and the comparisons with the other algorithms. In Section 6, the study's conclusion and recommendations for further research are provided.

2. Literature Review

In module-based product families (MBPF), products are derived by adding/removing different components to/from the platforms [27]. In the past decade, many researchers have developed tools and methods to enable product platforms, PFD, and assembly lines to offer cost-effective product variety and customization in MBPF. Many optimization approaches have been utilized, in the literature, for obtaining the optimal formation of the product platforms and designing assembly/disassembly processes aiming at minimizing the total production cost while producing a variety of products.

In this study, the different optimization techniques used to optimize the platform formation and balancing of the assembly line in MBPF are classified into three main types: exact, heuristic, and meta-heuristic. Exact methods typically use a bounded number of steps to find a proven optimal solution to an optimization problem. However, in most cases, they are time-consuming, particularly when solving large problems, such as ones with numerous dimensions, complicated constraints, multiple modes, or uncertainty [28]. The term heuristic is used for methods that find solutions from among all possible ones but offer no guarantee of obtaining the optimal solution. Typically, they are computationally efficient and require much less time than the exact methods [29]. Meta-heuristic methods usually start with random solutions and no information about the problem being solved and can also search for the optimal solutions within very large candidate solutions in the search space [30]. A taxonomy of such optimization methods applied to solve both platform formation and assembly line problems is presented in Fig. 1 and further reviewed in the following sub-sections.

2.1. Platform formation

The ability to have high product variety and short processing/lead times presents a competitive benefit to manufacturing systems. Industrial companies that endeavor for this ability prefer to reduce the internal variety between components (i.e., simplify the production processes) and increase the external variety among the obtained final products [31]. Grouping components and products into families is essential to identifying platforms for product families and ensuring a flexible product platform, which can be developed by applying clustering and commonality analysis. In their survey, Jose and Tollenaere [32] reviewed various methods that were applied to design platforms for product family problems. In their conclusion, they stated that the objective of the formation of product platforms is to decrease/increase the separate/common components in the established platforms and, at the same time, increase distinctiveness between the derived product variants. Constructing effective platforms contribute to successfully constructing a reconfigurable manufacturing system with 1) shorter lead time [33], 2) higher customer satisfaction [34], and 3) faster reaction to the markets characterized by uncertain demand and unforeseeable forecasting [35]. Over a long time, active work has been performed in developing exact, heuristic, and meta-heuristic optimization methods to design MBPFs with single and multiple objectives formulations [27].

In the literature, many optimization techniques have been applied to maximize or minimize a single objective MBPF. Examples of such single objectives are maximizing the commonality between members of product families, maximizing the overall system profitability, and minimizing the total cost.

2.1.1. Exact methods for MBPF

Several exact methods have been applied to find the optimal design of MBPF. A branch and bound (B&B) has been applied to solve MBPF using a mixed-integer programming method [36] aiming to find the best configurations of both module combinations and attributes through

multiple products. In the same work, sequential quadratic programming (SQP) has been integrated to calculate the lower-bound value of the objective function for each B&B node by solving its associated relaxed problem. In [37], dynamic programming (DP) has been applied to maximize a product family's total profit by considering the modules' interdependences at both product and product family levels. However, the application of exact methods is found to be applicable only to solve simple manufacturing systems with a limited number of products because the computational time inclines to increase exponentially with the system complexity, which usually relies on the number of components and the number of feasible assembly and disassembly operations [38]. This means that exact methods are inappropriate for a complex system, leading to more dependence on heuristic and meta-heuristic algorithms, which can deal with arbitrarily complex systems since the computational time increases polynomially, rather than exponentially, with the complexity [38].

2.1.2. Heuristic and meta-heuristic techniques for MBPF

According to [27], in 80% of adopted methods for identifying platforms, the functional decomposition is accomplished by heuristics, while 20% of these methods adopt meta-heuristics linked to optimization problems. In [39], a systematic heuristic method to find modules of a product from a functional model has been provided by introducing 3 heuristics of dominant flow, branching flow, and conversion, namely transmission function chains, to increase product modularity and innovation. In [40], the cascading target methodology has been extended for optimal product development to increase the shared components across all products. In their work, the single-product formulation has been altered to adapt the existence of common systems, subsystems, and/or components and locally established objectives. Recently, Gauss, Lacerda [41] proposed a module-based machinery design method aiming at supporting the design of the conceptual and system-level of modular machine families for reconfigurable manufacturing systems. However, their work was limited to the mechanical engineering domain.

Moreover, many meta-heuristics have been implemented in the literature to solve MBPF problems by increasing the commonality between the components used in the production processes. Some techniques are based on genetic algorithms (GAs), such as, in [42], a GA is applied to optimally divide a design structure matrix (DSM) to form common platforms for complex products. Also, to define the optimal set of common components to be shared between a group of products, the DSM, functional structure matrix (FSM), and GAs have been used, and an impact metric has been proposed [43]. In [44], the problem of product variety design has been mathematically formulated as a 0–1 integer-programming problem and an optimization algorithm based on a simulated annealing (SA) algorithm has been proposed to minimize the total cost for producing all presented products. A mathematical model and a GA have been proposed by Ben-Arieh, Easton [12] to increase commonality through various products. In their work, single and multiple platforms have been customized to differentiate the final product variants by assembling or disassembling components to the platforms. However, their model considers only product families and platforms, ignoring their assembly lines.

In PFD, decision-making for defining a strategy for the platform design can be viewed as a multidisciplinary optimization problem in terms of several factors, such as design variables, production cost, company profits, and customer satisfaction [9]. Therefore, many optimization techniques have been proposed to find the most compromise solution for a product family among multiple conflicting criteria by capturing the Pareto solutions and determining the optimal values of design variables for the platform and final product variants [45]. Several meta-heuristic methods have been applied to solve the module-based product family design as a multi-objective optimization problem [46]. A genetic algorithm (GA) has been applied to identify common and unique design variables for product families by formulating the problem

as multiple objectives to find the trade-off between commonality and performance [47]. A multi-objective particle swarm optimization (PSO) algorithm has been applied to determine the optimal design variables for the best platform design strategy based on the product family's commonality and design variation [9]. Other meta-heuristics have been based on the integration between the optimization techniques and commonality indices. For example, in [48], the authors developed a methodology using two decision techniques for achieving optimal product architecture. In their methodology, they first applied the analytic network process (ANP) to incorporate the designers' knowledge in calculating the relative importance of components regarding customer needs. Then, the goal programming approach was applied to incorporate the result of ANP and cost budget limitation to decide the platform. Although their technique employed the integration concept between techniques, in their construction way of the product family, they considered the weight and consistency of the components without considering the design cost itself.

Many research gaps in the design of product platforms and families have been identified in several studies [9]. Among them, developing new robust optimization models is still required further to enhance the efficiency of product families and platform design and to expand the number of common components. So, it is necessary to propose new methods to increase the commonality of components in platforms while manufacturing various products.

2.2. Assembly line

In the production plan of a reconfigurable manufacturing system [49], the assembly and disassembly are the most important factors/processes for determining the cost of the final products, as the costs of these customization processes are traditionally in the range of 26–51% of the overall cost of production/manufacturing [50]. Also, the percentage of workers involved in assembly/disassembly operations ranges from 21% to 61% [50]. Although several concepts and approaches have been introduced for optimizing the mixing process between various components and platforms to derive the final product variant, there is still a lack of an algorithm that can consider the design and planning of assembly/disassembly processes simultaneously with the formation of the product platforms.

Many exact, heuristic, and meta-heuristic algorithms have been introduced in the literature with two main objectives: 1) minimizing the cycle time (product's time-to-market) and 2) minimizing the total production cost. In [51], a lower bound exact method was introduced to solve a robotic production system's assembly line balancing problem. The lower bound used chain decomposition to explore the graph dependencies (precedence constraints between the tasks). Another exact method, the upper bound, has been introduced to solve assembly lines with uncertainty in the interval for operation times [52]. However, the algorithms used problem-specific dominance rules, which means their solutions techniques depend on these rules, and any changes in them will require a change in the solution method.

2.2.1. Heuristic techniques for assembly line

For more promising solutions, a two-stage heuristic was developed to solve the mixed-model assembly line balancing problem [53]. Their model's objectives were to reduce the number of workstations along the production line for a given cycle time and balance the workloads between the identified workstations. A hybrid algorithm of two heuristics has been used to solve the balancing assembly lines problem with heterogeneous workers while considering job rotation schedules [54]. In their work, mixed integer programming (MIP) was applied to choose appropriate schedules from a pool of heuristically structured solutions based on MIP neighbourhoods. A local search was used as a post-optimization method. Using phylogenetic networks, AlGeddawy and ElMaraghy [55] introduced an innovative design approach to develop an assembly line layout for delayed product differentiation

(DPD). The resulting cladogram identified the points of DPD and resembled the physical assembly system layout and was validated by using a family of electric kettles. Integration of components into parts was first developed in [15], where the author proposed a model from two basic steps by using Cladistics software to cluster the common components to define a core platform and apply physical commonality to enhance the produced core platform. Also, the authors of [7] proposed a DSS that applies MJPN to determine the best configuration of platforms of the PFD. Although their DSS showed a reduction in the platforms (internal) variety of their used case study, it also increased the assembly/disassembly costs. Recently, Hossain, Chakraborty [56] proposed a bi-level leader-follower joint optimisation model to propose a sustainable modular product architecture to minimize assembly complexity and discard the product. One of the study's significant flaws is that it only builds modular product architecture (PA) for a single product.

2.2.1.1. MJPN classification tool. Tools and concepts such as cladistics, phylogenetic networks (MJPN), generic bill of materials (GBOM), cost models, and others can assist in finding robust design results if they are associated with other technical purposes/objectives such as commonality and manufacturing flexibility [9]. Over time, the description of phylogenetic networks has remained to evolve because the first concept of the unity of species initiated by Darwin has a huge number of interpretations. Phylogenetic networks can be generally defined as any type of network that is utilized to represent potential evolutionary paths

in any sequence's type (e.g., mitochondrial DNA) [13]. These networks can be grouped into two main types: unrooted and rooted.

The Median-Joining Phylogenetic Network (MJPN) is a branch of unrooted phylogenetic networks. MJPN is originally utilized to trace and categorize DNA sequences based on their relationship to supposed family/ancestral nodes (median vectors) [13]. For comprehensive literature on phylogenetic networks, the readers are referred to [57] and for more information about the working principle of MJPN [13]. An example of rooted phylogenetic networks is Cladistics which is commonly employed in biology to explore the progression of living beings by mapping their evolution path and introducing their classification tree [58]. Cladistics was initiated as a hierarchical classification procedure with the capability of developing nested clusters of parts or products. ElMaraghy, AlGeddawy [59] presented this procedure in the manufactured artifacts and applied it to distinguish the similarities and differences between automobile engine variants and provided a graphical clustering representation, namely a cladogram, of these variants.

2.2.2. Meta-heuristic techniques for assembly line

A multi-objective assembly line rebalancing problem has been tackled by a GA to find a rebalancing of a line that achieves the best trade-off between productivity, re-balancing cost, and flexibility after reconfiguration [60]. In [61], a tabu search (TS) was developed to tackle the multi-manned assembly lines for reducing the number of workers and open multi-manned workstations for a predefined cycle time given several assignment constraints. A multi-objective PSO has been introduced to maximize both production rate (i.e., reduce the cycle time) and workload smoothing (i.e., evenly distribution of the workload to the workstations) in a simple assembly line balancing problem [62]. However, the author did not include very critical factors in their objective functions, such as costs and/or profits. A DE method has been presented to solve multi-objective nuclear engineering fuel assembly design problems for maximal plutonium content and minimum power peaking factor [63]. Although optimization algorithms often provide improved solutions in a reasonable computational time, they are extremely problem-dependent and involve extensive tuning, which decreases their flexibility to tackle more applications/problems, and hence their applicability to the real world is reduced.

2.2.3. DE for assembly line

Because of its simple structure, robustness, speed, and ease of use, among meta-heuristics, DE is considered one of the best evolutionary algorithms (EAs) for tackling such optimization problems [26]. DE is a meta-heuristic method that belongs to the family of EAs. It is a stochastic population-based search method that was initially introduced by [64] to solve continuous optimization problems where real numbers represent the decision variables of the solutions. DE has been widely applied to solve optimization problems in different areas and has several practical applications [65,66]. Moreover, it has also been applied to solve several permutation-based optimization problems in discrete domains. Recent discrete versions of DE have been successfully developed to tackle instances of combinatorial optimization problems (COPs), such as resource-constrained projects [67,68], travelling salesman problems (TSPs) [69], travelling thief problems (TTPs) [70] and many others with very promising results. So, DE has been considered an effective method for solving COPs. However, further features, such as its convergence speed and avoiding becoming trapped in local optimum solutions, are still required to be enhanced to improve its performance while tackling complex COPs.

2.3. Motivation and contributions

Motivated by the identified shortcomings of the platform formation, assembly lines techniques, and standard DE, we introduce a reactive framework that integrates the MJPN classification tool with the DE optimization technique for reducing the total production costs by

Table 1

Sets, indices, variables and parameters in the proposed Packaged-based model and MJPN-DE algorithm.

Sets	
L	Set of platforms
C	Set of components
S	Set of product variants to be produced
P	Set of packages
A	Set of added components in each platform
Indices	
i	Index for platform
j	Index for product variants
n	Index for components
p	Index for packages
Variables	
$x_{j,n}$	A binary variable to show components configuration for each product variant
$s_{j,n}$	A binary variable to show assembled components in each product variant
$d_{j,n}$	A binary variable to show disassembled components in each product variant
$w_{p,n}$	A binary variable to show components in each package
$a_{p,j}$	A binary variable to show packages in each product variant
Parameters	
PL_{max}	Total number of platforms
V	Total number of product variants
N	Total number of components
Q	Maximum number of components per package
NC_p	Number of components per package p
NP	Total number of packaged components
NU	Total number of separate components
r	Received discount on the component's original price
P_c	Discounted price per component
f_p	Frequency of using package p in all the product variants
f_{U_n}	Frequency of using component n in all the product variants
ac_j	Assembly and labour costs for producing product variant j
dc_j	Disassembly and labour costs for producing product variant j
uc_n	Fixed purchasing cost of component n
$\gamma_{n,j}$	Required cost to assemble component n to product j
$at_{n,j}$	Required time to assemble component n to product j
$\partial_{n,j}$	Required cost to disassemble component n to product j
$dt_{n,j}$	Required time to disassemble component n to product j
cr	Crossover rate
F	Mutation rate
PS	Population size
φ	Percentage of PS solutions to apply the proposed heuristic method to them

simplifying the required production processes by increasing the number of components to be packaged and hence decrease the number of assembly/disassembly processes in a configurable PFD.

Three contributions are to be considered in this study:

- A technical development of an integrated framework that combines the MJPN classification tool and DE algorithm.
- Introducing a new package-based model aiming at reducing both the assembly/disassembly stations/points and cycle time by considering the concept of packages instead of separate components.
- Proposing an enhanced version of the DE evolutionary algorithm to improve the solutions resultant from the MJPN algorithm through employing the proposed package-based model.

3. Problem formulation and mathematical model

This section defines the proposed package-based mathematical model. The problem is to assemble/disassemble the fewest possible parts/components to produce the requisite number of product variants at the minimum overall manufacturing cost.

In the proposed model, we apply a two-stage process. In the first stage, we apply the MJPN algorithm through the “Network.10” tool available at <https://www.fluxus-engineering.com/sharenet.htm>. The input for the MJPN is a binary matrix (x_{jn}), which shows the relationship between all the components and product variants. The output from the MJPN’s stage is a phylogenetic network that shows/classifies the number of product platforms, components, and variants. Using the obtained classification, the components that can be assembled, disassembled, and packaged are indicated. All these outputs are utilized as input for the second stage, where the proposed package-based mathematical model is applied to enhance the resultant solution from the MJPN algorithm and obtain the required number of product variants while minimizing the material purchasing, assembly, and disassembly costs.

The notations of different sets, parameters and variables applied in the proposed packaged-based model and MJPN-DE algorithm are presented in Table 1. In the following sub-sections, the mathematical notations are further explained through each stage.

3.1. Stage 1 – MJPN-related notations

A binary variable (x_{jn}), which shows the components configuration for each product variant, is used as an input to the ‘Network 10’ tool to produce the Phylogenetic Network by applying the MJPN algorithm.

$$x_{jn} = \begin{cases} 1 & \text{if component } n \text{ exists in product } j \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where x_{jn} is a binary variable that indicates if components n is used to produce product variant j .

The output from the MJPN algorithm is a Phylogenetic Network that results in the following sets:

1. Set of product platforms (L) = $[1, 2, \dots, i, \dots, L_{\max}]$, where i is an index and $L_{\max}$ is the total number of platforms.
2. Set of product variants (S) = $[1, 2, \dots, j, \dots, V]$, where j is an index and V is the total product variants.
3. Set of components (C) = $[1, 2, \dots, n, \dots, N]$, where n is an index and N is the total components.
4. Paths (production lines) for producing each product variant, where a set of platforms and separate components are listed for each product variant (S_j).

3.2. Stage 2 – Proposed package-based mathematical model

After identifying the above sets from the resulting Phylogenetic

Network, the mathematical model can be defined based on the following decision variables and considering the following assumptions:

3.2.1. Decision variables

The decision variables, which define the assembled ($s_{j,n}$), disassembled ($d_{j,n}$) and packaged ($w_{p,n}$ and $a_{p,j}$), where $p \in [1, 2, \dots, p, \dots, P]$ and P is the total number of packages, components in each product variant j , are shown in Equations (2)–(5), respectively.

$$s_{j,n} = \begin{cases} 1 & \text{if } n \text{ component is assembled for product variant } j \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$d_{j,n} = \begin{cases} 1 & \text{if } n \text{ component is disassembled for product variant } j \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

$$w_{p,n} = \begin{cases} 1 & \text{if } n \text{ component is involved in package } p \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

$$a_{p,j} = \begin{cases} 1 & \text{if package } p \text{ is involved to produce product variant } j \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

3.2.2. Assumptions

The following assumptions are considered in the proposed mathematical model:

Assumption 1. (Assembly cost of product platforms):

The assembly cost is calculated for all components used for producing each product variant (S_j) from the start. In the case of using product platforms, which contain one or more components, the assembly cost for adding a specific product platform (i.e., L_i) to produce a product variant (i.e., j) is calculated as the sum of the assembly costs of all the components in L_i . If L_i is used for producing another product variant (i.e., $j+1$), no assembly costs will be added to setup L_i again. This assumption is formulated in this model as constraint no. 5, Eq.(12).

Assumption 2. (Components in packages):

When dealing with packages, we assume that the components in each package are already assembled by the supplier and required no extra assembly costs (i.e., the supplier is providing each package with its components already assembled and ready to use). To use a specific package for producing a product variant, the assembly/disassembly cost for adding/removing that package to/from the product platform will be calculated as the assembly/disassembly cost of the first component only in that package. This assumption is formulated in this model through constraint no. 6, Eq.(13).

Assumption 3. (all-unit discount policy):

Purchasing components in packages will receive a $r\%$ discount on the component’s original price (uc_n) for each added component in a package. The total discount is based on the number of components per package (NC_p).

$$\text{Price per component } (P_c) = uc_n \times (1 - r)^{NC_p}, \forall n \in N, \forall p \in P. \quad (6)$$

This equation shows that for every added component in the package, the manufacturer will receive a $r\%$ discount from the supplier. As a result, maximizing the number of components in packages will lead to lower total purchasing costs.

Assumption 4. (module-based PFD):

We are assuming that the proposed mathematical model applies to manufacture systems with module-based PFD and is not suitable for

scalable-based ones.

Some other assumptions are considered when dealing with packages, such as:

1. The NC_p is limited to be equal to or less than Q , where Q is the highest number of components per package. This is applied through constraint no. 9, Eq.16.
2. There must be at least two components in each package (constraint no. 7, Eq.14).
3. The components in packages cannot be used as separate components in the whole product life cycle for all the product variants or product platforms (constraint no. 8, Eq.15).

3.2.3. Objective function and Constraints

The objective function can be mathematically formulated as minimizing the total value resulting from the following four components with the same weight/importance applied to each.

The main objective function is to minimize the total production cost by achieving the following objectives:

3.3. Minimize the fixed purchasing cost of materials/components by

- i. Purchasing most of the components in packages ($NP = \sum_{p=1}^P NC_p$) will activate the all-unit discount policy and hence reduce the total purchasing cost.
- ii. Purchasing the rest of the components ($NU = N - NP$) as separate (unpacked components).
 - a. Minimizing the assembly and disassembly costs (costs and labour) required to produce the product variants.

The mathematical formulation can be presented as follows:

Minimize:

$$\begin{aligned} & \sum_{p=1}^P \left(\sum_{v=1}^V a_{p,v} \times \sum_{n=1}^{NC_p} p_{c,p,n} \right) + \sum_{n=1}^{NU} \left(\sum_{j=1}^V s_{n,j} \times uc_n \right) + \sum_{j=1}^V \\ & \times \sum_{n=1}^N s_{n,j} \cdot (\gamma_{n,j} + at_{n,j} \delta_{n,j}) + \sum_{j=1}^V \sum_{n=1}^N d_{n,j} \cdot (\partial_{n,j} + dt_{n,j} \mu_{n,j}), \forall p \\ & = 1, \dots, P, \forall n = 1, \dots, N, \forall j = 1, \dots, V. \end{aligned} \quad (7)$$

Subject to:

$$\sum_{j=1}^V s_{j,n} \leq \sum_{j=1}^V x_{j,n}, n = 1, \dots, N, \quad (8)$$

$$\sum_{j=1}^V d_{j,n} \cdot x_{j,n} \leq 0, n = 1, \dots, N, \quad (9)$$

$$\sum_{j=1}^V d_{j,n} \leq \sum_{j=1}^V s_{j,n}, n = 1, \dots, N, \quad (10)$$

$$\sum_{j=1}^V s_{j,n} \times \sum_{n=1}^N s_{j,n} \geq 1, j = 1, \dots, V, n = 1, \dots, N, \quad (11)$$

$$ac_{j,n} = \sum_{n=A_n+1}^N s_{n,j} \cdot (\gamma_{n,j} + at_{n,j} \delta_{n,j}), j = 1, \dots, V, \forall n \notin A, \quad (12)$$

$$a_{p,j} \leq w_{p,n}, j = 1, \dots, V, p = 1, \dots, P, n = 1, \quad (13)$$

$$\sum_{n=1}^{NC_p} w_{p,n} \geq 2, p = 1, \dots, P, \quad (14)$$

$$w_{p,n} + s_{n,j} + d_{n,j} = 1, p = 1, \dots, P, n = 1, \dots, N, j = 1, \dots, V, \quad (15)$$

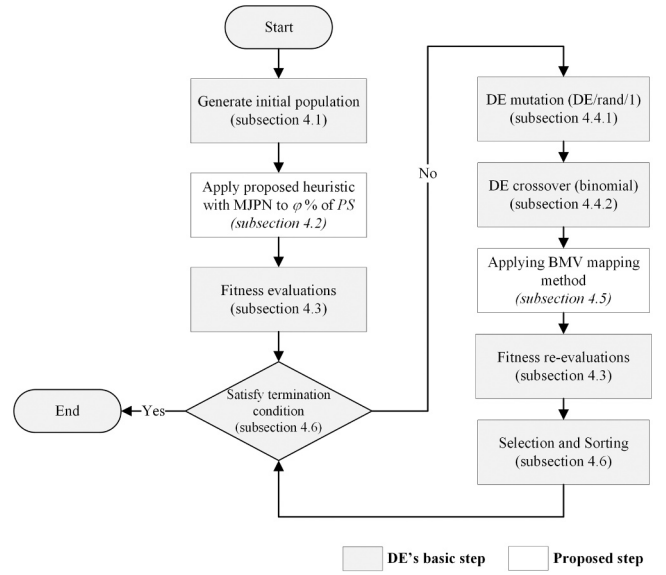


Fig. 2. Flowchart of the proposed MJPN-DE algorithm.

$$NC_p \leq Q, \quad (16)$$

$$s_{j,n}, d_{j,n}, w_{p,n}, a_{p,j} \geq 0, \quad (17)$$

where the first part of the objective function shows the purchasing cost of all packaged components where the frequency of using package p in all the product variants (V) is calculated as $\sum_{v=1}^V a_{p,v}$. $a_{p,j}$ a binary decision variable that equals 1 if package p is used to produce product variant j , and 0 otherwise, NC_p the total number of components in package p , and $p_{c,p,n}$ the discounted purchasing cost of each component n in each package p . The second part shows the purchasing cost of all unpackaged components where the frequency of using component n in all the product variants (V) is calculated as $\sum_{j=1}^V s_{n,j}$. $s_{n,j}$ is a binary variable to detect if components n is assembled in product j or not. NU is the total number of components that cannot be packaged with other components and can be computed as the difference between the total number of components N and the packaged components NP , and uc_n the cost of purchasing a separate (unpacked) component n . The third part shows the assembly and labour costs where $\gamma_{n,j}$ and $at_{n,j}$ are the required cost and time to assemble component n to product j , respectively and $\delta_{n,j}$ the assembly labour cost per hour to assemble component n to product j . The last part shows the disassembly and labour costs where $d_{n,j}$ is a binary variable to detect if components n is disassembled in product j or not. $\partial_{n,j}$ and $dt_{n,j}$ the required cost and time to disassemble component n to product j , respectively and $\mu_{n,j}$ the disassembly labour cost per hour to assemble component n to product j .

In our mathematical model, a solution is regarded as feasible if it meets the specified set of constraints presented in Eqs. (8) – (17). The purpose of each constraint is further explained as follows:

Constraint (8) ensures that for a specific product variant (j), only components that are existing in that j 's configuration can be assembled by ensuring that the total number of components to be assembled ($s_{j,n}$) to j must be lower than or equal to the total number of components in the original (input) configuration ($x_{j,n}$) of that product variant. Constraints (9) and (10) guarantee that for a specific j , components can be disassembled only if they are not included in the original (input) configuration (i.e., $x_{j,n} = 0$) of that j , and already exists in the assembled components (i.e., $s_{j,n} = 1$). Constraint (11) ensures that each product variant must contain at least one component and that all the purchased components are used. Constraint (12) ensures that the assembly costs of the components in any product platform are calculated only once by

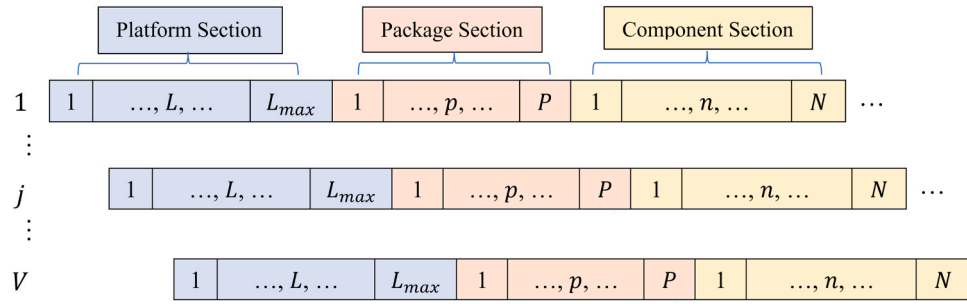


Fig. 3. Example of solution representation in the proposed DE algorithm.

calculating the assembly cost for the first component directly after the last component of the last added product platform. Constraint (13) guarantees that the assembly/disassembly costs of packages are calculated as one integral part (or component). Constraint (14) ensures that the number of components in any package must be at least two. Constraint (15) ensures that any components coming in a package (packaged components) must not be used as stand-alone/separate components (unpackaged components) in any product variant. Constraint (16) guarantees that number of components is not exceed the maximum number of components allowed per package. Finally,

Constraint (17) is the non-negative constraint that enforces that $s_{j,n}$, $d_{j,n}$, $w_{p,n}$, $a_{p,j}$ decision variables must be greater than or equal to zero.

4. Proposed MJPN-DE framework

This section introduces the proposed MJPN-DE framework. The proposed framework, its design, and essential elements are presented in Fig. 2 and further explained in the following sub-sections.

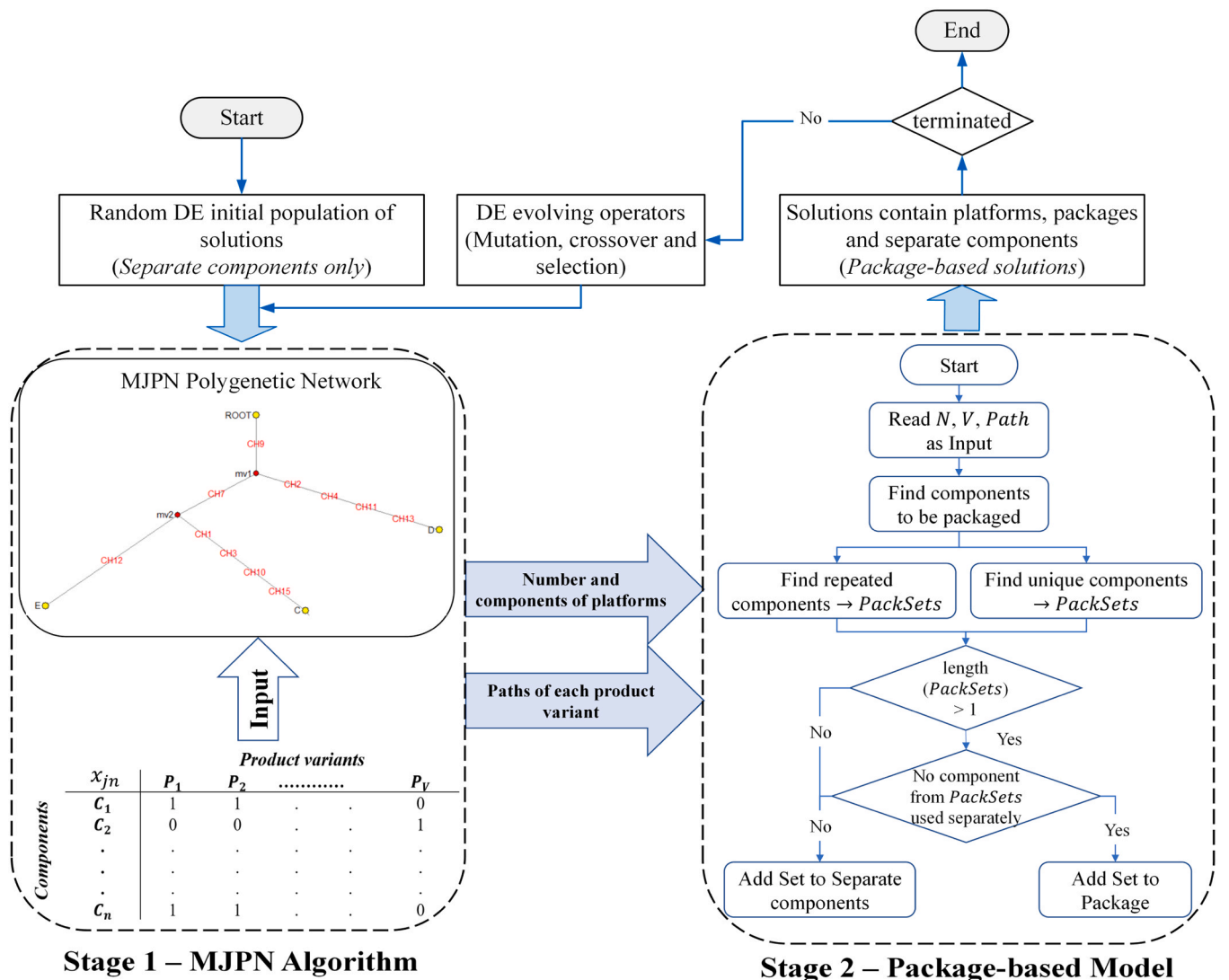


Fig. 4. Outline of the proposed integrated framework.

4.1. Solution representation and initial population

In the proposed DE, each candidate solution is encoded as a binary vector. Each vector contains all the product variants (i.e., $1, \dots, j, \dots, V$) that are created by a specific PFD. In the proposed solution representation, three sections for platforms, packages (integrated components), and separate components with a length $L_{\max}$, P and N , respectively, are presented for each product variant. The total length of each vector that represents one product variant equals $BigN = L_{\max} + P + N$, while the total length of each solution that includes all the product variants equals $BigN \times V$.

In this binary representation and for each product variant, the value 1 indicates that the index (*Ind*) of this value is involved in the production process of that product variant. The location of *Ind* in the different sections indicates if it is a platform, package or a separate component. For example, If $0 < Ind \leq L_{\max}$, then the chosen index is a platform, else if $L_{\max} < Ind \leq L_{\max} + P$, then it is a package, else if $L_{\max} + P < Ind \leq BigN$, then it is a separate component. An example of one vector/solution with V product variants is given in Fig. 3.

4.2. Implementation of the proposed package-based model

In the proposed MJPN-DE framework, MJPN is first applied to initiate the product family design phase. In this phase, MJPN identifies the distinct product platforms, their number, and structure, and shows the potential assembly and disassembly processes of components for each candidate solution in the initial population. Such processes, which are required to transform each platform into a product variant by gaining and losing components to customise them, are usually involving setup cost and time. The number of product platforms along with their components, the number of separate components, and the assembly and disassembly processes, obtained from the MJPN classification tool (in stage 1), are then used as inputs to the proposed package-based model (in stage 2) to further classify the produced components into packaged and separate components. The integration between the MJPN and the proposed package-based model in the proposed MJPN-DE framework is

allow more diversity in the initial population.

In Fig. 4, after the generation of the initial population and employing the MJPN, the package-based model is applied for constructing packages of components by integrating components that are most frequently employed together for the production of specific or numerous product variants. Then, the DE algorithm uses the constructed packages, instead of separate components, to complete the setup (assembly/disassembly) processes and calculate their production costs. Applying the package-based model will also help the decision-makers to decide which components to be purchased as separate and which as packages and hence obtain benefits from the “all-unit discount policy” to reduce the purchasing costs as well.

In its implementation, the list of components used to differentiate each product variant and identified by the MJPN classification tool are employed to construct the new packages. A sequence of set operations (i.e., intersections and differences) between the identified list of components within the product variants are performed in order to determine the commonality of each component (i.e., common, shared, unique). Algorithm 1 shows how to put the proposed heuristic technique into practice. To locate shared components utilized in the production process of two or more product variants, the intersection of the first product's set and all other product sets, then the intersection of the second product's set and all other remaining products, and so on is calculated. The resulting set/collection of components is labelled as a package, as shown in steps 6, 10, and 12 in Algorithm 1. To find the unique components that are only used in one product version, the set difference is used. As in the intersection process, the difference between each set of components and all other sets of other products is calculated, and the resultant set of components is employed as a package (steps 7 and 19 in Algorithm 1).

Two main conditions must be satisfied for each package to be created:

1. Each package must contain at least 2 components.
2. Components in each package can be used neither as separate components nor as a part of any other package.

Algorithm 1 - Pseudo-code of the implementation of the proposed model – Packaging process

```

1.  $N, V, L_{\max}, MJPN_{path} \leftarrow$  Read data from the MJPN
2. for  $i = 1$  to  $L_{\max}$ 
3.    $L_{mat}(i) \leftarrow$  Components of platform  $i$ 
4. end for
5.  $Pac \leftarrow MJPN_{path}$  with no repetitive components,  $t \leftarrow 1$ ,  $Count \leftarrow 1$ 
6. for  $l = t+1$  to  $V$ 
7.    $RepPack_l \leftarrow$  intersection between  $Pac_l$  and  $Pac_t$  % Find repeated sets of packages.
8.    $UniPack_l \leftarrow$  set difference between  $Pac_l$  and  $Pac_t$  % Find unique sets of packages.
9. end for
10. if  $length(RepPack) > 0$  then //Locate repeated sets
11.   for  $w = 1$  to  $length(RepPack) - 1$ 
12.      $FRepPack_w =$  intersection( $RepPack_w$  and  $RepPack_{w+1}$ )
13.   end for
14.    $Count \leftarrow Count + 1$ 
15.    $Fin\_Pack_{count} =$  set difference between  $RepPack$  and  $FRepPack$ 
16. end if
17. if  $length(UniPack) > 0$  then //Locate unique sets
18.    $u \leftarrow 1$ 
19.   while  $u \leq length(UniPack) - 1$ 
20.      $Check \leftarrow$  if any components of  $UniPack_u$  is separately used in  $MJPN_{path}$ .
21.     if  $Check == 0$ , then
22.        $Count \leftarrow Count + 1$ 
23.        $Fin\_Pack_{count} \leftarrow UniPack_u$ 
24.     end if
25.      $u \leftarrow u + 1$ 
26.   end while
27. end if

```

shown in Fig. 4. The produced solutions from stage 2 are then used as candidate solutions in the DE's population for further enhancements and

To continue the production processes, the identified/created packages will be called to be assembled or disassembled as one part, not separate components (i.e., the separate components that have been combined into packages will be replaced with the identified packages). The replacement process will be executed on both product platforms and separate component levels. In Algorithm 2, the pseudo-code for replacing the packaged components with their packages is presented. Each product variant's solution is split into three segments, as indicated in Fig. 3, for platforms, packages, and individual/separate components. The components are replaced with packages in both the platforms and separate components sections (Algorithm 2, steps 5–12 and 18–24), while the used packages are indicated in the package section (Algorithm 2, steps 13–17). Any repeated components indicate that these components need to be removed/disassembled from the assembly line of a specific product variant. The disassembling process is done in steps 26–28 in Algorithm 2. By applying this integration of components, it is expected that the total production, purchasing costs and the setup time to produce the products will be significantly reduced [71].

Any solution that violates one or more constraints is considered infeasible. For each infeasible solution, the penalty function-based constraint handling technique [72] is used. From these techniques, the method of adaptive penalties (AP method), which is considered the most efficient way by [72], is used by adding a penalty value to the total cost of production that increases as the violation value of this solution increases.

4.4. Perturbation operators

As in all evolutionary algorithms, mutation and crossover are the main operators to enable the exploration process of the search space. In this study, as the product platforms and packages sections (in Fig. 3 of the solution representation) are determined by the MJPN tool and further enhanced by including packages using the proposed package-based heuristic method, respectively, the mutation and crossover will be applied only to the separate components' section (Section 4 in the solution representation). The applied mutation and crossover operators are further discussed in the following sub-sections.

Algorithm 2 Pseudo-code of the proposed model - Replacing the components by packages in all product variants

```

1.   $P \leftarrow \text{Count}$  //number of identified packages
2.   $\text{BigN} \leftarrow L_{\max} + P + N$ ,  $s, d, a \leftarrow \text{zeros}$ 
3.  for  $j=1$  to  $V$  do
4.      for  $n=1$  to  $\text{BigN}$ 
5.          if  $n \leq L_{\max}$  // Platform section
6.              if  $\text{all}(L_{\text{mat}}(n))$  is used in  $\text{MJPN}_{\text{path}}(j)$  then
7.                  if  $\text{any}(\text{Fin\_Pack}(n))$  is used in  $L_{\text{mat}}(n)$  then
8.                       $p \leftarrow n$ 
9.                       $a(j, p) \leftarrow 1$ 
10.                      $s(j, \text{Fin\_Pack}(n)(1)) \leftarrow 1$ 
11.                  else
12.                       $s(j, n) \leftarrow 1$ 
13.                  end if
14.              end if
15.          elseif  $L_{\max} > n \leq L_{\max} + P$  // Package section
16.              if  $\text{all}(\text{Fin\_Pack}(n))$  is used in  $\text{MJPN}_{\text{path}}(j)$  then
17.                   $s(j, n) \leftarrow 1$ 
18.                   $p \leftarrow n - L_{\max}$ 
19.                   $a(j, p) \leftarrow 1$ 
20.              end if
21.          elseif  $L_{\max} + P > n \leq \text{BigN}$  // Separate components section
22.              if  $\text{MJPN}_{\text{path}}(j, n)$  is not member in any  $\text{Fin\_Pack}(n)$ , or  $L_{\text{mat}}$  then
23.                   $s(j, n) \leftarrow 1$ 
24.              elseif  $\text{any}(\text{Fin\_Pack}(n))$  is used in  $\text{MJPN}_{\text{path}}(j, n)$  then
25.                   $p \leftarrow n - (L_{\max} + P)$ 
26.                   $a(j, p) \leftarrow 1$ 
27.                   $s(j, \text{Fin\_Pack}(n)(1)) \leftarrow 1$ 
28.              end if
29.              if  $n > 1$ , then
30.                   $\text{check} \leftarrow$  if a component in  $\text{MJPN}_{\text{path}}(j, n-1)$  has been repeated in either  $\text{MJPN}_{\text{path}}(j, n)$ ,
31.                   $\text{Fin\_Pack}$  or  $L_{\text{mat}}$ , then
32.                      if true then // disassemble this component
33.                           $d(j, n) \leftarrow 1$ 
34.                      end if
35.                  end if
36.              end if
37.          end for
38.      end for

```

4.3. Fitness Evaluation

To determine which solutions should be retained for the next generation and which should be removed from the population, the quality of each solution in the population is measured. This is usually achieved by a pre-given objective function. In this paper, the fitness values of the candidate solutions are calculated using the objective function in Eq.(7). The feasibility of each solution is determined by satisfying the

4.4.1. Mutation operator

In DE, a mutation operator is applied to improve the diversity of the candidate solutions in the population and thus allow DE to explore new areas in the search space. In the proposed DE, the simplest strategy, 'DE/curr-to-rand/1', is applied using Eq.(18) to explore the search space of the problem. For every individual, DE/curr-to-rand/1' mutation generates a new offspring by calculating a scaled difference vector between two other individuals, usually randomly selected from the current

population, and adding it to the mutated individual.

$$\vec{v}_{i,g+1}^j = \vec{x}_{i,g}^j + F \times (\vec{x}_{r_2,g}^j - \vec{x}_{r_3,g}^j), r_2 \neq r_3 \neq i, \quad (18)$$

where $\vec{v}_{i,g+1}^j$ is the produced mutant vector of product j ($= 1, 2, \dots, N$) in solution i ($= 1, 2, \dots, PS$) and generation g . F is a scale factor that rules the amplification of the differential variation between the two vectors $\vec{x}_{r_2,g}^j$ and $\vec{x}_{r_3,g}^j$, determining how distant the new offspring should be from the mutant solution $\vec{x}_{r_3,g}^j$. The value of F usually lies within the range of $[0,2]$ [73]. $\vec{x}_{i,g}^j$, $\vec{x}_{r_2,g}^j$, $\vec{x}_{r_3,g}^j$ are three mutually distinct individuals selected at random from the population and are not equal.

4.4.2. Crossover operator

In the proposed DE, for each product variant j , the binomial version of the crossover is used to generate offspring from other solutions parents randomly selected from the mutant vectors according to CR defined in Eq.(19). In the binomial crossover, a new offspring can be obtained by including some genes (a component of solution) from a mutant vector

least chosen from the mutant vector $\vec{v}_{i,g+1}^j$ [64].

4.5. Mapping method

As the learning concept of the mutation operator in DE uses the Euclidean distance, the produced trial vectors usually have continuous variables, whereas the objective function of our discussed problem requires variables with binary values to evaluate the fitness of each solution. To overcome this issue, the solutions of the newly generated trial vectors are mapped to their equivalent binary variables using the best-matched value (BMV) mapping method, which is proposed in [74] for solving discrete problems and further extended in [26] for solving binary ones. In this paper, the population of solutions with continuous variables will be used for operating the mutation and crossover in the next generations, whereas the mapped solutions will be used for evaluating their qualities and selecting the ones with the best fitness values. The main steps of the applied BMV mapping method are listed in Algorithm 3. In this paper, we set all probabilities of BMV as recommended in [26].

Algorithm 3. BMV mapping method's main steps [26].

C	Characteristics	Platforms			Packages			
		L_1	L_2	L_3	p_1	p_2	p_3	p_4
C_1	Body: plastic	-	-	✓	✓	-	-	-
C_2	Body: metal	-	-	-	-	-	-	✓
C_3	Handle: plastic	-	-	✓	✓	-	-	-
C_4	Handle: metal	-	-	-	-	-	-	✓
C_5	Boiling checker	-	-	✓	-	✓	-	-
C_6	On-off switch	-	-	✓	-	✓	-	-
C_7	Temperature control unit	-	✓	-	-	-	-	-
C_8	Side coil unit	-	-	-	-	-	✓	-
C_9	Bottom coil unit	✓	-	-	-	-	-	-
C_{10}	Door unit	-	-	✓	✓	-	-	-
C_{11}	Steam valve	-	-	-	-	-	-	✓
C_{12}	Burner surface	-	-	-	-	-	-	-
C_{13}	Detachable base	-	-	-	-	-	-	-
C_{14}	Base plug	-	-	-	-	-	✓	-
C_{15}	Body plug	-	-	✓	-	-	-	-

✓ Indicates that these components are used in L_3 as p_1 packaged components

✓ Indicates that these components are used in L_3 as p_2 packaged components

($\vec{v}_{i,g+1}^j$) that is randomly selected with probability CR , while the rest of the genes are taken from the current target vector (original solution before mutation) with probability $1 - CR$, as shown in Eq.(19). The ($j = e_j$) condition ensures that the mutant vector is sharing with at least one gene to obtain the offspring.

$$\vec{u}_{i,g+1}^j = \begin{cases} \vec{v}_{i,g+1}^j & \text{rand}(j) \leq CR \text{ or } j = e_j \vec{x}_{i,g}^j \\ \vec{x}_{i,g}^j & \text{otherwise,} \end{cases} \quad (19)$$

where $\vec{u}_{i,g+1}^j$ is the new offspring, usually called a trial vector, of product variant j in solution i of generation g . CR is the crossover possibility in the range of $[0,1]$, $\text{rand}(j)$ is a uniform random number generator within $[0,1]$ that generates a random number for each j^{th} evaluation and e_j is a randomly chosen dimension to guarantee that one element of $\vec{u}_{i,g+1}^j$ is at

4.6. Selection, sorting and termination conditions

At the end of each iteration and to continue the iterations of the DE's evolutionary process, a comparison between the solutions of the new (after applying mutation, crossover, and mapping) and old (from the previous iteration) populations is conducted in terms of the fitness values of each solution. In order to achieve that, the greedy selection strategy is applied to guarantee the survival of the individuals with the best fitness to the next generation. In the newly selected population, the individuals with the worst fitness values are replaced by the survival individuals in both binary and continuous populations to ensure the simultaneous update of both populations to be used during the next iterations. Finally, both populations, which comprise solutions with the best fitness values, are decided and then sorted based on their fitness values, where the fittest solution ranked first, and so on.

Table 2
Components in product variants.

Steps	Computational Complexity
Read Input Date	N
Determine components in platforms	N
Distribute platforms and components to the proposed matrix	$N^2 + \log N$
Find components to be packaged	$N^2 + N + \log N + N \log N + 2 \log N$
Replace components with packages	$2N^2 \log N + 2N^3 \log N$
Calculate the costs	$N^2 + 2 \log N$

$$\vec{x}_{i,g+1} = \begin{cases} \vec{u}_{i,g+1}, & f(\vec{u}_{i,g+1}) < f(\vec{x}_{i,g}) \\ \vec{x}_{i,g}, & \text{otherwise.} \end{cases} \quad (20)$$

The processes of the DE's evolutionary operations are terminated if the number of generations exceeds a pre-defined limit (Fit_{max}).

5. Experimental Results

This section starts by showing the computational complexity of the proposed package-based model for solving the Module-based PFD in subsections 5.1. In subsections 5.2 and 5.3, an explanation for the case studies used in this study is provided. The parametric analyses to set the best values for the DE's parameters and the sensitive analysis of two essential parameters are shown in Subsection 5.4. In subsection 5.5, a numerical result that shows how the proposed package-based model can be used in such a manufacturing system to reduce the total cost of the production process is given. Finally, Subsections 5.6 and 5.7 present the efficacy of the proposed Package-based model and proposed MJPN-DE algorithm, respectively, by conducting several comparisons to evaluate their performance against existing models and state-of-the-art algorithms.

In order to assess the proposed MJPN-DE algorithm and its performance, computational experiments were carried out to solve 1) a real-life case study of a Kettle product family with five final products to be manufactured from 15 separate components and 2) a large-sized simulated case study where 30 various products are made from 100 different components. The proposed MJPN-DE was written in MATLAB R2019b and ran on a PC with a Core i7 processor and 16 GB of RAM. In our experiments, each instance was run 30 times independently, with 100, 500, and 1000 generations for the case study I and 1000, 5000, and 10000 generations for case study II in each run.

Table 3
Components in product variants.

C	Characteristics	Product variants				
		A	B	C	D	E
C_1	Body: plastic	1	1	1	0	0
C_2	Body: metal	0	0	0	1	0
C_3	Handle: plastic	1	1	1	0	0
C_4	Handle: metal	0	0	0	1	0
C_5	Boiling checker	1	1	0	0	0
C_6	On-off switch	1	1	0	0	0
C_7	Temperature control unit	0	0	1	0	1
C_8	Side coil unit	1	0	0	0	0
C_9	Bottom coil unit	0	1	1	1	1
C_{10}	Door unit	1	1	1	0	0
C_{11}	Steam valve	0	0	0	1	0
C_{12}	Burner surface	0	0	0	0	1
C_{13}	Detachable base	0	1	0	1	0
C_{14}	Base plug	1	0	0	0	0
C_{15}	Body plug	0	1	1	0	0

5.1. Complexity of the proposed model

The big O notation [75], which is used to classify algorithms while employed to solve a certain problem based on how their running time grows as the input size expands, is utilized in this paper. The big O is generally used to analyse an algorithm's efficiency and gives the algorithm complexity in terms of input N . Also, it gives a method to abstract the efficiency of the code regardless of the capability of the machine they run on (machine-independent), as it examines the basic computer steps of the code. To calculate the complexity of the implementation of the proposed mathematical model to solve the product family design problem in RMS, the computational complexity of the different steps of the proposed algorithm has been measured as shown in Table 2.

By combining the big O notations for the different steps of coding the proposed algorithm, the overall computational complexity can be computed as follows:

$$\begin{aligned} \text{Complexity} = & N + N + N^2 + \log N + N^2 + N + \log N + N \log N + 2 \log N \\ & + 2N^2 \log N + N + 2N^3 \log N + N + N^2 + 2 \log N. \end{aligned}$$

$$\text{Complexity} = 2N^3 \log N + 2N^2 \log N + 3N^2 + N \log N + 6 \log N + 3N.$$

$$\text{Complexity} = O(N^3 \log N) = O(N^3).$$

Fig. 5 shows the problem's complexity by using the mathematical equation of the big O notation ($f(x) = O(g(x))$), where $g(x)$ refers to the

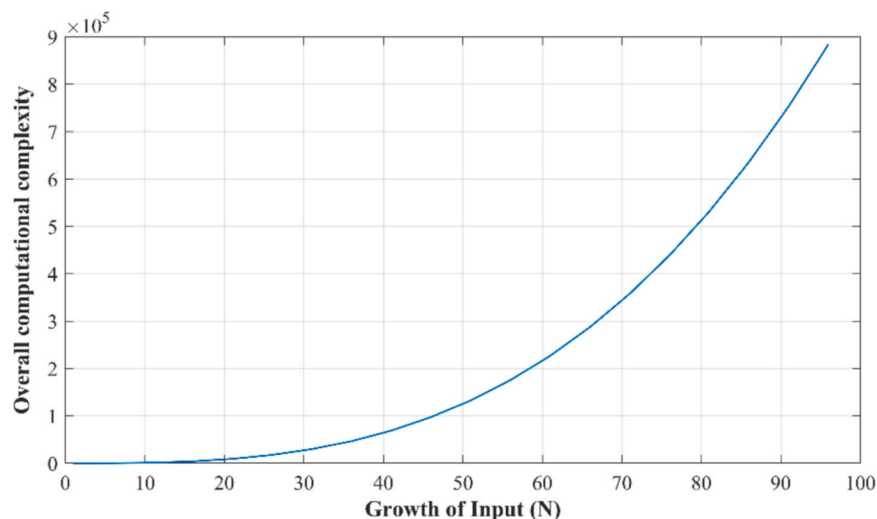


Fig. 5. Overall computational complexity.

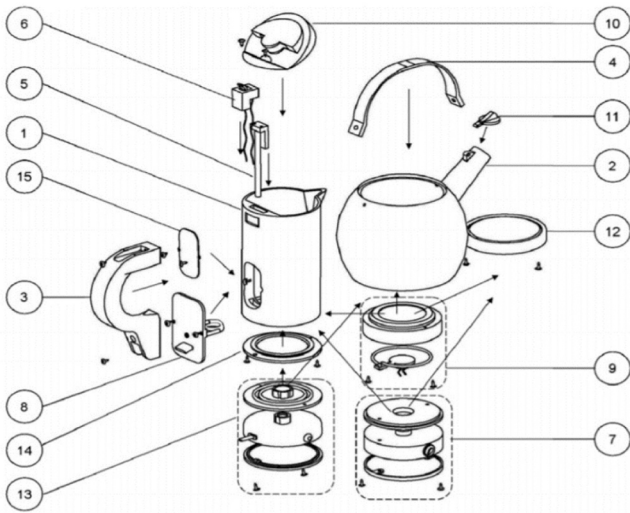


Fig. 6. Exploded view of components of studied Kettle product family as presented in [55].

Table 4
Parameter values utilized in the implementation's upper and lower bounds.

Components	Symbol	Lower	Upper	Unit
Cost per component				
Purchasing	uc	1	100	\$
Assembly	γ	1	100	\$
Disassembly	δ	1	100	\$
Packaging	σ	1	100	\$
Labour cost per hour				
Assembly	δ	1	25	\$
Disassembly	μ	1	25	\$
Packaging	ρ	1	25	\$
Time required per component				
Assembly	at	1	1000	minute
Disassembly	dt	1	1000	minute
Packaging	pt	1	1000	minute

computed overall computational complexity and x to real numbers. Here, x equals N .

Fig. 5 indicates that solving the proposed model using exact methods may be unsuitable as it will take a very large amount of computational time before getting the solution.

5.2. Case study I

The numerical case study is presented to demonstrate the model's ability to solve real-world situations. We have 15 components/characteristics (C_1 to C_{15}), in a Kettle product family, which are used to make 5 different kettle product variants (i.e., A, B, C, D, E), with the components (n) required to make each product variant (pv_j).

Table 3 lists the component parameters for each product variant ($x_{j,n}$) and Fig. 6 graphically shows them. The company needs to select which components can be packaged and which should be kept separate, as this might increase or decrease the costs of both component purchases and manufacturing processes. All other factors/inputs, such as component

Table 5
Combination of parameter values.

Parameters	Levels				
	1	2	3	4	5
PS	50	100	150	200	250
CR	0.1	0.3	0.5	0.7	0.9
F	0.2	0.4	0.6	0.8	1

Table 6
Orthogonal table and total cost values.

Experiment No.	Parameter Combination			Total Cost (average)	
	PS	CR	F	Case study I	Case study II
1	50	0.1	0.2	3119.51	130662.27
2	50	0.3	0.4	3170.44	130655.41
3	50	0.5	0.6	3210.07	130653.24
4	50	0.7	0.8	3210.07	130643.35
5	50	0.9	1.0	3210.07	130615.13
6	100	0.1	0.4	3568.11	129399.22
7	100	0.3	0.6	2843.59	129398.12
8	100	0.5	0.8	3568.11	129396.67
9	100	0.7	1.0	2843.59	129394.73
10	100	0.9	0.2	2843.59	129375.81
11	150	0.1	0.6	3036.29	128297.21
12	150	0.3	0.8	3036.29	128286.86
13	150	0.5	1.0	3036.29	128275.47
14	150	0.7	0.2	2178.55	128275.47
15	150	0.9	0.4	2765.17	128902.37
16	200	0.1	0.8	3339.51	131387.69
17	200	0.3	1.0	3339.51	131383.89
18	200	0.5	0.2	3339.51	131354.39
19	200	0.7	0.4	2216.49	130374.00
20	200	0.9	0.6	2216.49	131348.43
21	250	0.1	1.0	4752.88	132040.30
22	250	0.3	0.2	3828.5	132018.93
23	250	0.5	0.4	5001.88	132011.58
24	250	0.7	0.6	4674.88	131999.19
25	250	0.9	0.8	4457.45	131998.34

Table 7
Design Summary of Taguchi method.

Taguchi Array	L25(5 ³)
Factors	3
Runs	25

purchasing costs, assembly/disassembly costs, and required time, were created at random, while the discount rate (r^0) was set to 10%. The upper and lower values of all parameters used in the implementation are shown in Table 4.

5.3. Case study II

To validate the model's ability to deal with real-world problems with larger dimensions, a case study that contains 100 components (c_1 to c_{100}) to produce 30 different product variants (A, B, C, ..., AC, AD) has been generated based on a simulated scenario and used to be solved by the proposed MJPN-DE and other comparative algorithms. As per the limitation of the page size, the details of case study II, including the relationship matrix between components and product variants ($x_{j,n}$ matrix) and the Phylogenetic tree resulting from the MJPN tool are published online and can be accessed through this link². Both cases have been solved by the proposed MJPN-DE and other comparative algorithms such as Non-platform, MJPN, standard GA, and DE.

The parameter tuning, sensitivity analysis, and the obtained results from solving Case studies I and II are presented in the following subsections.

5.4. Parameter tuning and sensitivity analysis

5.4.1. DE's parameters tuning

The proposed MJPN-DE has 3 key parameters, namely: population size (PS), crossover rate (CR), and mutation factor (F), that could influence the quality of the achieved results. Table 5 shows the different values of each parameter. The Taguchi design approach, which is a

² Uploaded Case study II.xlsx (sharepoint.com)

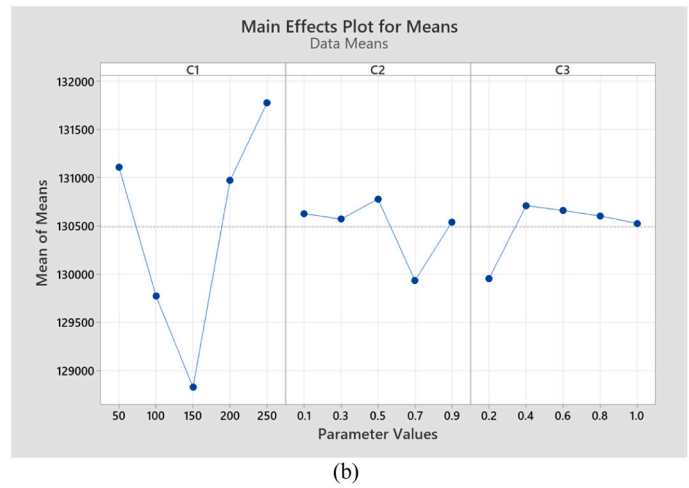
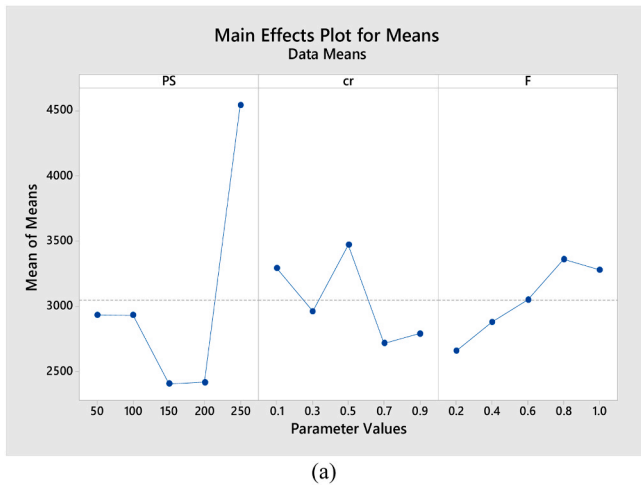


Fig. 7. Proposed algorithm's factor level trend is based on the average total production cost of (a) Case study I and (b) Case study II.

Table 8
Response Table for Means.

Level	Case study I			Case study II		
	PS	CR	F	PS	CR	F
1	2937	3294	2662	131106	130628	129952
2	2935	2963	2882	129773	130570	130710
3	2407	3473	3052	128827	130778	130659
4	2420	2718	3363	130970	129933	130603
5	4543	2792	3282	131774	130540	130526
Delta	2136	755	701	2946	845	758
Rank	1	2	3	1	2	3

popular method that uses an orthogonal array, is employed with L25 as the orthogonal design to determine the optimal combination of these parameter values. To examine all of the orthogonal array combinations, 25 sets of tests were designed for solving case studies I and II. For each experiment, the proposed DE was executed for up to 30 runs with 1000 generations for each. For each parameter combination, the proposed algorithm has been applied to solve the same case study using the same initial population (a fixed seed of random generator has been used to ensure generating the same initial solutions). The best-obtained production costs are listed in Table 6, and the design summary of the used Taguchi method is presented in Table 7.

In Fig. 7, the values of each parameter obtained from solving both case studies (I and II) are presented. Both sub-figures in Fig. 7 show that the best combinations of parameter values can be decided as $PS=150$, $CR=0.7$, and $F=0.2$ and that these parameter settings are suitable for

solving the targeted module-based PFD problem with small and large dimensions/sizes.

The impact of each parameter on the MJPN-DE's performance is presented in Table 8 based on the obtained values of the production cost from both case studies (I and II). In both cases, the table shows that the performance is mostly affected by PS as it is ranked first, while F has the lowest impact. In Table 8, Delta is the difference between the maximum and minimum average response values for each parameter. The ranks are assigned based on Delta values; for example, the highest Delta value is Rank 1, the second highest is Rank 2, and so on, to indicate the relative effect of each parameter on the response.

5.4.2. Sensitivity analysis of the number of packages in each product variant

To demonstrate the efficiency of the proposed MJPN-DE algorithm, the number of packages used in each product variant has been analysed by using the original number of packages (obtained by solving the proposed package-based model using the proposed MJPN-DE) and start to reduce this number by eliminating package by package and notice the changes in the value of the total production cost. Fig. 8 represents the changes in the total production cost trend while reducing or increasing the number of packages. From Fig. 8, we can notice that the cost value of the production process with no applied packages equals the cost value produced from MJPN, and this value is greatly reduced by adding each package until reaching the obtained cost value from the proposed MJPN-DE. Basically, the trend shows the major differences in the cost values before and after using the packages of components.

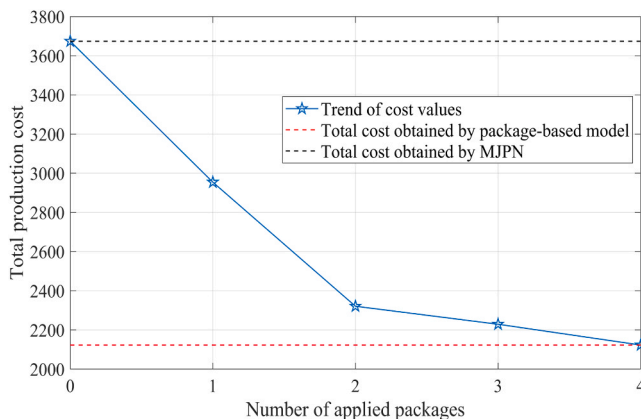


Fig. 8. Number of packages against total production cost.

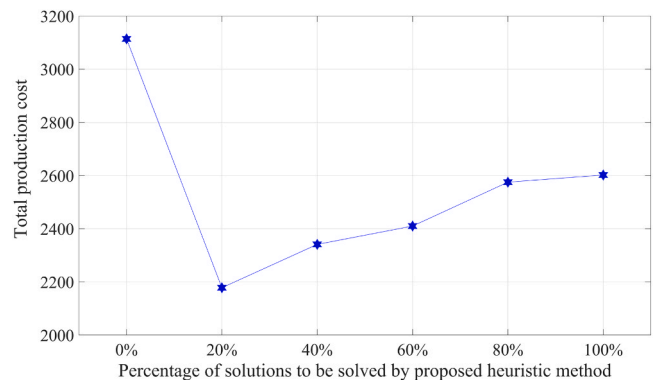


Fig. 9. Effect of the number of solved individuals by the heuristic from the population on the total cost value.

Table 9
Identified platforms and packages and their components in case study I.

C	Characteristics	Platforms			Packages			
		L ₁	L ₂	L ₃	p ₁	p ₂	p ₃	p ₄
C ₁	Body: plastic	-	-	✓	✓	-	-	-
C ₂	Body: metal	-	-	-	-	-	-	✓
C ₃	Handle: plastic	-	-	✓	✓	-	-	-
C ₄	Handle: metal	-	-	-	-	-	-	✓
C ₅	Boiling checker	-	-	✓	-	✓	-	-
C ₆	On-off switch	-	-	✓	-	✓	-	-
C ₇	Temperature control unit	-	✓	-	-	-	-	-
C ₈	Side coil unit	-	-	-	-	-	✓	-
C ₉	Bottom coil unit	✓	-	-	-	-	-	-
C ₁₀	Door unit	-	-	✓	✓	-	-	-
C ₁₁	Steam valve	-	-	-	-	-	-	✓
C ₁₂	Burner surface	-	-	-	-	-	-	-
C ₁₃	Detachable base	-	-	-	-	-	-	-
C ₁₄	Base plug	-	-	-	-	-	✓	-
C ₁₅	Body plug	-	-	✓	-	-	-	-
✓	Indicates that these components are used in L ₃ as p ₁ packaged components							
✓	Indicates that these components are used in L ₃ as p ₂ packaged components							

Table 10
Numerical components of each product platform and packages.

Platforms				Packages		
No.	Symbol	Packages	Components	No.	Symbol	Components
1	L ₁	-	9	1	p ₁	1, 3, 10
2	L ₂	-	7	2	p ₂	5, 6
3	L ₃	p ₁ , p ₂	15	3	p ₃	8, 14
				4	p ₄	2, 4, 11

5.4.3. Sensitivity analysis of φ solutions

To give an indication of the efficacy of the introduced DE based on the proposed package-based model, the number of solutions (φ) to be solved by the proposed model (as a percentage of the PS) is examined in this subsection to determine the effect of the obtained solutions on the overall population's diversity compared with the randomly generated ones.

Table 11
Assembly lines and assembly/disassembly costs using Non-platform, MJPN, and Package-based.

Product variant	Technique	Setup (production) processes / Assembly Line	Number of assembly points
A	Non-platform	1→3→10→6→5→8→14	7
	MJPN	L ₁ → L ₂ → L ₃ →8→14→ - 9→ - 15	7
	Package-based	L ₁ → L ₂ → L ₃ →p ₃ → - 9→ - 15	6
B	Non-platform	9→1→3→10→15→6→5→13	8
	MJPN	L ₁ → L ₂ → L ₃ →13	4
	Package-based	L ₁ → L ₂ → L ₃ →13	4
C	Non-platform	9→7→1→3→10→15	6
	MJPN	L ₁ → L ₂ →1→3→10→15	6
	Package-based	L ₁ → L ₂ →p ₁ →15	4
D	Non-platform	9→2→4→11→13	5
	MJPN	L ₁ →2→4→11→13	5
	Package-based	L ₁ →p ₄ →13	3
E	Non-platform	9→7→12	3
	MJPN	L ₁ →L ₂ →12	3
	Package-based	L ₁ →L ₂ →12	3

In Fig. 9, the proposed MJPN-DE algorithm runs with φ set to different values of 0, 20, 40, 60, 80, and 100 (%) of PS. The results reveal that solving 20% of the candidate solutions using the proposed package-based model in the initial population can obtain a lower cost by 30.06% than $\varphi=0\%$ and 16.5% than the average cost obtained by the other parameter's values. The results illustrate the importance of the proposed model and its efficacy in developing product variants at lower costs. Also, due to the lack of diversity in the population, we discovered that larger values of φ increase the total cost value.

5.5. Numerical results

After determining the best parameter combination, the proposed DE algorithm was executed for 30 runs with 100, 500, and 1000 generations in each run with the best settings to solve the case study I with the following best/final results can be achieved.

The components used to form the product platforms, defined by MJPN, and packages of components that have been created by the proposed method by grouping several components into one integral part (package), are shown in Table 9. As shown in this table, the packages have been utilized to form some product platforms. For example, packages p₁ and p₂ have been used to form a platform L₃. This indicates that the packages can be useful for both platform and separate component levels. The numerical components of each product platform and package are listed in Table 10.

In Table 10, we can notice that the newly defined packages can contribute to reducing the usage of separate components not only in the assembly line (i.e., components to be assembled and disassembled) but also inside the defined platforms, which leads to minimizing the cost to establish such platforms. For example, in platform number 3 (L₃), instead of using components 1, 3, 10, 6, 5, and 15 to form the platform (as shown in Table 9), we used p₁, p₂ and component 15. Thus, the steps/processes of assembly were reduced by 50% (from 6 steps to 3).

5.6. Efficacy of the proposed package-based model

In this sub-section, many comparisons, based on case study I, have been conducted to assess the performance of the package-based model in the proposed MJPN-DE, against the MJPN and Non-platform models, in terms of 1) the number of assembly points, 2) assembly, disassembly, and purchasing costs, 3) frequency of components' usage and success rates of both cases and 4) postponement effectiveness.

5.6.1. Setup processes (assembly points)

The optimal production paths that use the combination of platforms, packages, and separate components to produce each product variant in the mentioned case study using the Non-platform, MJPN, and package-based models are shown in Table 11, where - 9 and - 15 mean that components 9 and 15 need to be disassembled from the assembly line to produce the desired product variant. In this table, it can be noticed that the production paths created by the proposed package-based model always have a minimum or at least equal assembly points compared with MJPN and Non-platform models. In such product variants, no packages are used in their production process, and hence, their assembly lines include only platforms and separate components. Such results indicate the importance of using packages of components in decreasing the length of the assembly line and hence the total production points.

5.6.2. Assembly, disassembly and purchasing costs

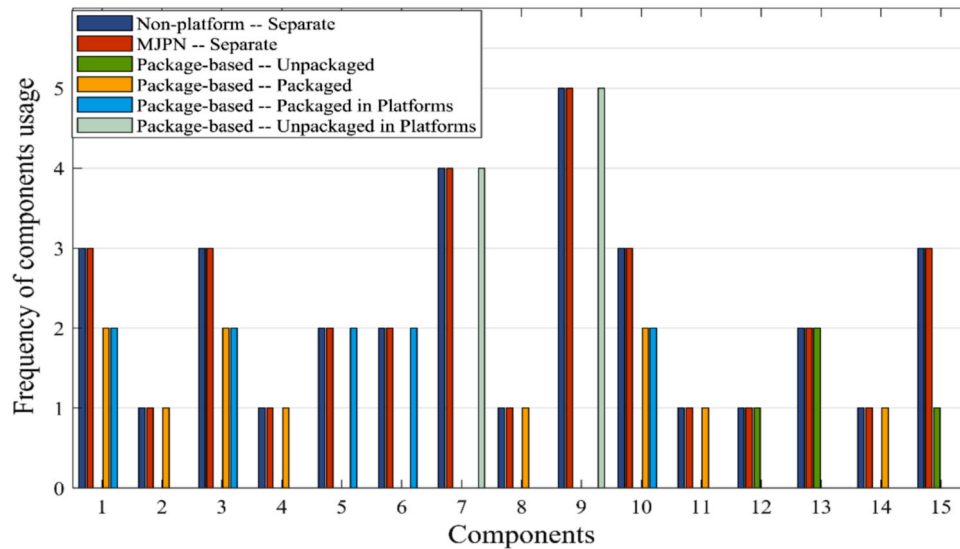
In Table 12, the overall product costs employing the Non-platform, MJPN, and Package-based techniques, as well as the assembly, disassembly, and purchase costs of components in packages (packaged) and separate (unpackaged) for each product variant, are shown.

The findings of the comparisons between the three techniques demonstrate that the proposed package-based model can save 57.5% of component purchase prices and 50.2% and 32.5% of assembly expenses,

Table 12

Costs of assembly, disassembly, and purchase.

Product variant	Technique	Costs				Total cost per variant (assembly +disassembly)
		Assembly	Disassembly	Purchasing (for all product variants)		
				Packaged	Unpackaged	
A	Non-platform	395.54	0	-	3661	395.54
	MJPN	325.94	63.0039	-	3661	388.94
	Package-based	180.02	63.0039	1141.9	722	243.0239
B	Non-platform	287.04	0	-	-	287.04
	MJPN	94.08	20.41	-	-	114.49
	Package-based	94.08	20.41	-	-	114.49
C	Non-platform	164.88	0	-	-	164.88
	MJPN	107.28	0	-	-	107.28
	Package-based	51.51	0	-	-	51.51
D	Non-platform	280.44	0	-	-	280.44
	MJPN	243.25	0	-	-	243.25
	Package-based	168.28	0	-	-	168.28
E	Non-platform	118.98	0	-	-	118.98
	MJPN	81.79	0	-	-	81.79
	Package-based	81.79	0	-	-	81.79
Total production cost (assembly + disassembly + purchasing costs)					Non-platform	4907.88
					MJPN	4596.75
					Package-based	2522.99

**Fig. 10.** Breakdown of components' usage to produce 5 product variants (case study I).

respectively, when compared to non-platform and MJPN-based approaches.

5.6.3. Frequency of components' usage

In Fig. 10, the frequency of the usage of each component and how it has been used (i.e., used as separate, unpackaged, unpackaged in the platform, packaged, packaged in the platform) are graphically presented. The main objective of this figure is to illustrate how the proposed package-based model can divide different components into different categories rather than use them as separate components only. In Fig. 10, we can notice that the unpackaged components in platforms (i.e., components 7 and 9) and the packaged 'unique' components (i.e., 2, 4, and 11 in p_4) have the same frequency usage with different models. On the other hand, packaged components that are used in both platforms and packages, such as components 1, 3, and 10 in p_1 have less frequency of usage compared with their usage as separate components. Although the frequency usage of each component to produce each product variant will always be fixed, using the maximum number of such components in packages can significantly reduce the assembly/disassembly processes among them, as such processes are not required to be done in packages, according to Assumption 2 in subsection 3.2.2, and thus reduce the costs

of the setup/production processes as well as the purchasing of materials/components.

The relative percentage deviation (RPD) is proposed by Ruiz, Maroto [76] and used in quantitative research to compare two numbers while taking into account the "sizes" of the entities being compared. Such a comparison is a unitless number that is stated as a ratio. In this study, the RPD of the values of the best and average runs obtained from the package-based method and the MJPN and Non-platform ones with 100, 500, and 1000 generations are shown in Table 13 and calculated using Eq.(21).

$$RPD = \frac{Sol_A - Sol_B}{Sol_A} \times 100, \quad (21)$$

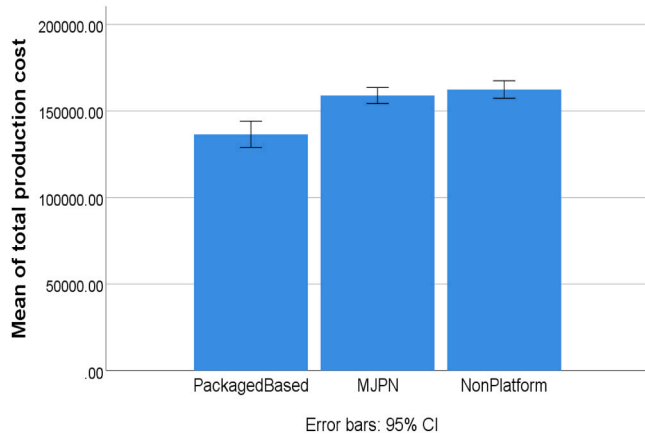
where Sol_A and Sol_B are the best solutions found by algorithms A and B, respectively.

The RPD values in Table 13 claim that in both small and large case studies, the proposed heuristic based on the package-based model directs the proposed MJPN-DE to obtain solutions with lower production costs than MJPN and non-platform models. Furthermore, the RPD values of the average runs reveal that the proposed model's performance

Table 13

RPD values (MJPN-DE based on package-based model vs. MJPN and non-platform).

Techniques	Case study I						Case study II			
	Best run			Avg. of 30 runs			Best run		Avg. of 30 runs	
Generations	100	500	1000	100	500	1000	1000	5000	1000	5000
Package-based vs. MJPN	3.44%	39.65%	29.16%	8.48%	40.27%	41.79%	15.29%	26.13%	13.80%	15.90%
Package-based vs. Non-platform	15.00%	36.61%	35.69%	9.85%	43.88%	45.48%	16.47%	27.53%	15.09%	17.20%

**Fig. 11.** Interval plot at 95% confidence of the mean production cost of the comparative techniques based on Cases I and II.

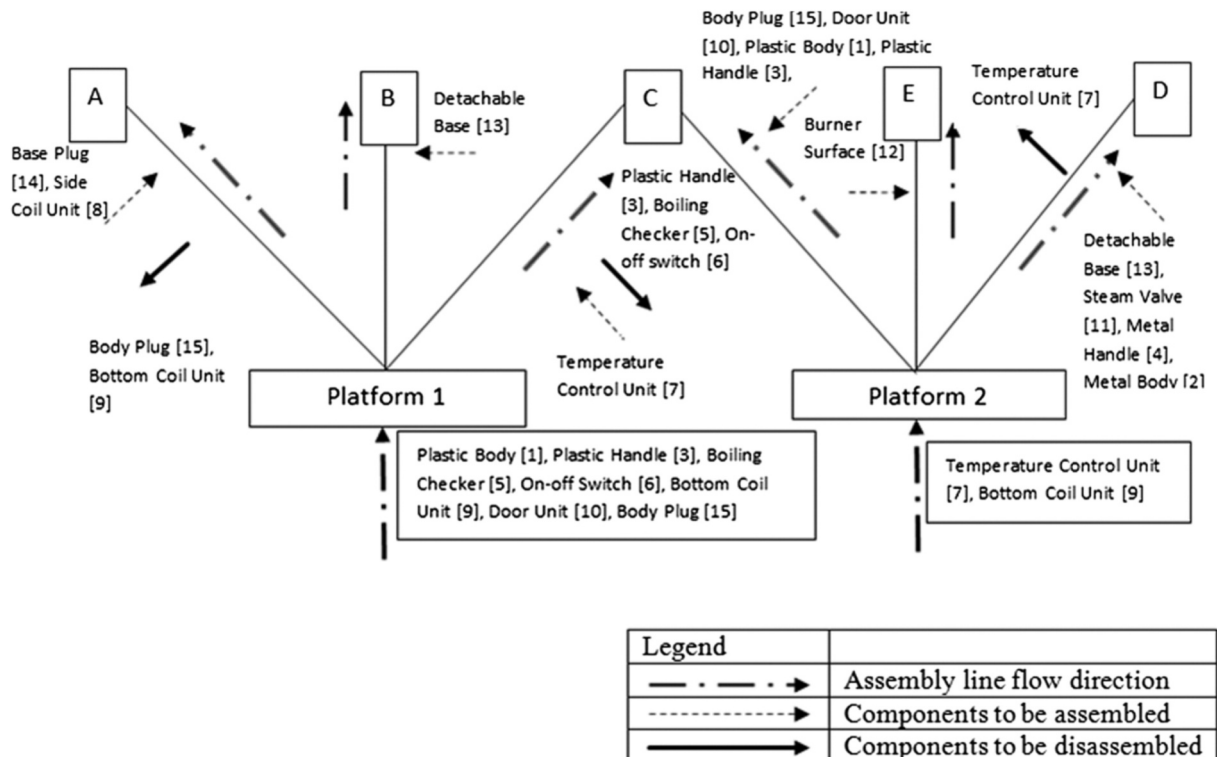
improves consistently as the generation numbers increase. To assess the significance of those RPD values, an interval plot at 95% confidence is presented in Fig. 11, where the mean production costs obtained by running package-based, MJPN and non-platform models 30 times/runs are shown in the y-axis. The error bars in Fig. 11 show that the package-based model achieved results that are significantly better than the other two comparative models.

5.6.4. Postponement Effectiveness (PE)

The effectiveness of the layout strategy (structure of the PFD) produced by the proposed package-based model in postponing product differentiation is compared with those achieved by the delay product differentiation (DPD) platform network model [55] and other layout strategies. To do that, the delayed differentiation metric proposed by [55] is applied in this paper. Postponement effectiveness is determined by measuring the ability of a particular structure/layout to increase the commonality between the components of this layout and hence delay the product differentiation points. The layout with the best postponement effectiveness is the one that has the maximum number of components in the main platforms and, at the same time, the minimum number of components to be assembled or disassembled to produce a certain product variant. The postponement effectiveness metric used is a function of 1) the number of components in each platform, 2) the number of components to be assembled/disassembled to/from platforms, 3) assembly/disassembly labour times, 4) quantity/number of products in a specific platform and 6) the number of products that without a platform.

$$PE = \frac{(\varepsilon + \sum_{j=1}^V f_j) \times \sum_{j=1}^V \sum_{n=1}^N Q_j (at_{n,j} s_{j,n} + dt_{n,j} d_{n,j})}{(\sum_{j=1}^V Q_j) \times (\sum_{L=1}^{L_{max}} \sum_{j=1}^V \sum_{n=1}^N at_{n,j} s_{j,n} P_{j,pl} U_{nj})} \times w, \quad (22)$$

where ε is the number of products not in a platform, f_j a binary variable

**Fig. 12.** Layout strategy obtained by DPD modular platform network model products as presented in [13].

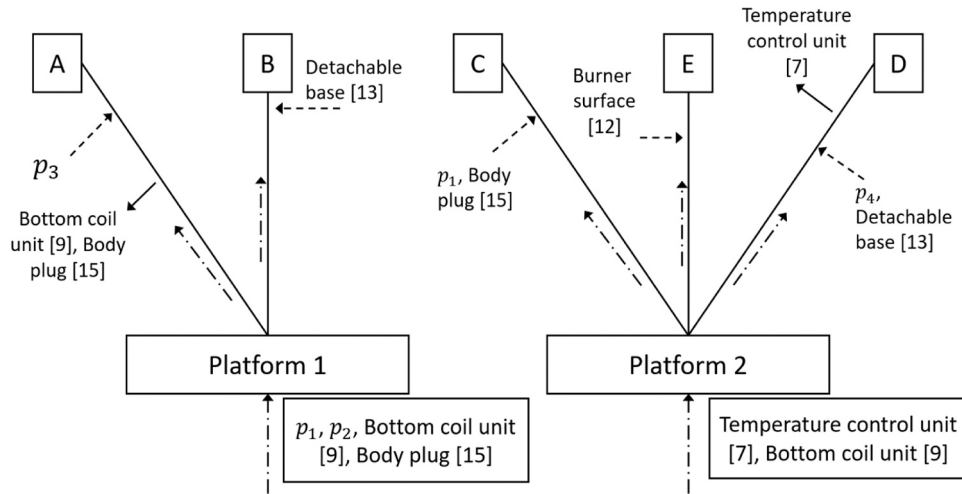


Fig. 13. Layout strategy obtained by applying the proposed package-based model.

that indicates if product j doesn't exist in a platform, $P_{j,pl}$ indicates if product j is made using platform L , and $U_{j,pl}$ is the quantity of platform L needed for product j .

From the above factors, it can be noticed that by increasing the number of components in a platform, the postponement effectiveness of the obtained platform is increased. The delayed differentiation metric with the previously mentioned factors is expressed in Eq. (22). Also, the author kept other unknown (additional future) factors as a constant and represented them with w . In Eq. (22), the method with a value of PE close to 0, means that this method produces more efficient platforms, and can effectively postpone the products differentiation point. The metric in Eq. (22) is applied to compare the layouts obtained from the proposed DE and other mechanisms.

Figs. 12 and 13 graphically compare the assembly line to produce the five different product variants using the 15 components by the DFD modular platform network model and the proposed package-based model, respectively. Two main differences between the two models can be quickly noticed.

- First is the number of separate components to be assembled or disassembled, as in Fig. 12, the DFD model assembles and disassembles 15 components to the defined platforms to differentiate the product variants compared with only 10 components assembled and

disassembled by the proposed package-based model in Fig. 13. This means that the proposed model can save up to 33.33% of the assembly and disassembly processes that are required to obtain the different product variants.

- The second difference is that the DFD model can produce product variant j in two different ways (i.e., either from Platform 1 or Platform 2), while the proposed package-based model produces variant j using only Platform 2. This is because the package-based model cannot use Platform 1 to produce variant v as the production process requires disassembling component number 3 (which is part of package 1 (p_1)) from Platform 1. According to constraint no. 8 in Eq. 15, component number 3 cannot be used as a separate component and disassembling it from Platform 1 (and hence from p_1) will directly violate one of the constraints of packages' forming.

Moreover, Eq. (22) is applied to calculate the postponement effectiveness (PE) of the DFD model, the proposed package-based model, and a model which uses a cladistic tool with assembly only. The resultant PE values are shown as follows:

- *Package-based model*

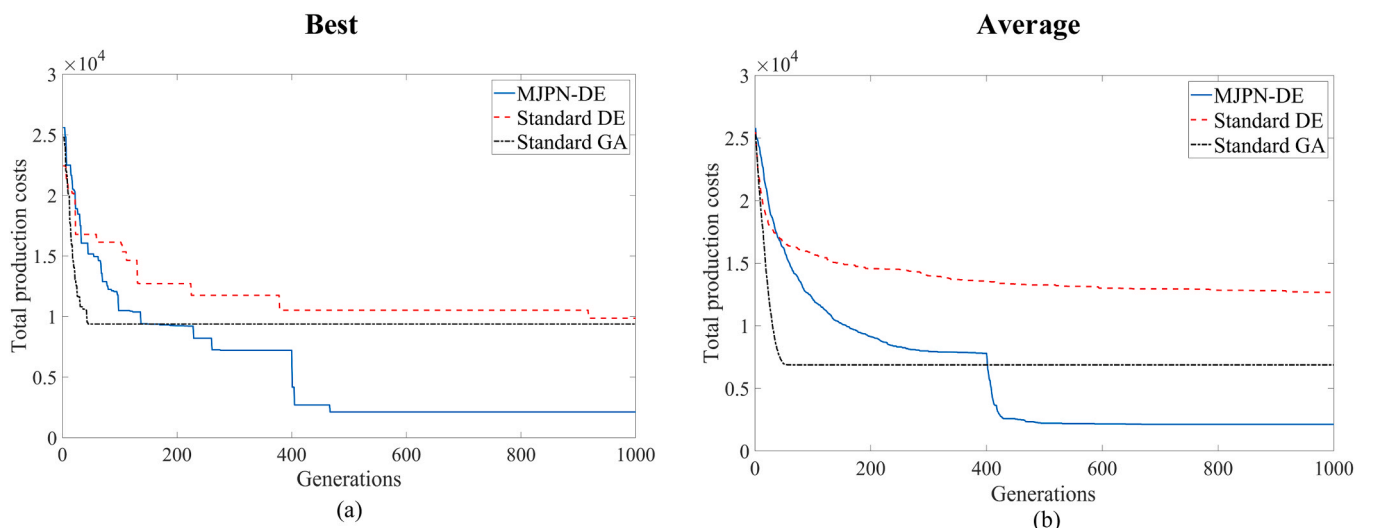


Fig. 14. Convergence plot for best (a) and average (b) runs of standard GA, standard DE and proposed MJPN-DE through 1000 generations – Case study I.

$$PE = \frac{(1+1)(3 \times 100 + 1 \times 200 + 2 \times 500 + 1 \times 50)}{(100 + 200 + 500 + 50)((100 + 200) \times 7) + ((500 + 50) \times 2)} w$$

$$= 0.0011397w.$$

- *DPD modular platform network model*

$$PE = \frac{(1+1)(4 \times 100 + 1 \times 200 + 4 \times 500 + 1 \times 50)}{(100 + 200 + 500 + 50)((100 + 200) \times 7) + ((500 + 50) \times 2)} w$$

$$= 0.0019485w.$$

- *Cladistics (using platform customization by assembly only)*

$$PE = \frac{(1+3)(100 + 200)}{(100 + 200)((100 + 200) \times 3)} w = 0.0044w.$$

The results confirm that the proposed package-based model can effectively produce layouts with better postponement effectiveness and can outperform other existing methods. Although the authors of the *PE* metric reported that the DPD modular platform network and Cladistics models achieved *PE* with values $0.000487w$ and $0.048w$, according to our calculations, these values are wrongly calculated and the corrected ones should be $0.0019485w$ and $0.0044w$, respectively.

5.7. Efficacy of the proposed MJPN-DE algorithm

In this sub-section, the performance of the proposed integrated MJPN-DE is algorithmically compared with those of the standard versions of GA and DE for solving both case studies using the defined

mathematical model and the proposed package-based model.

5.7.1. Standard GA and DE

We are referring to “Standard” as a “simple modelling” of GA and DE while tackling permutation-based optimization problems [77]. Both case studies were solved by two standard well-known evolutionary algorithms: Genetic Algorithm (Standard GA) and DE (Standard DE). One-point crossover and flip mutation have been utilized in Standard GA, while binomial crossover and the simplest type of DE mutation (namely: DE/rand) have been employed in Standard DE. According to [77], the crossover and mutation probabilities were adjusted to 0.7 and 0.2, respectively, and both algorithms implemented the greedy selection technique.

Comparisons between the Standard GA, Standard DE, and proposed MJPN-DE in terms of convergence towards the optimal solutions while solving case studies I and II, are given in Figs. 14 and 15, respectively. For case study I, all the algorithms were executed for 30 runs with 1000 generations in each run, while for case study II, with 1000 and 5000 generations. In Fig. 14, the results of the best run after each generation value have been represented on the left-hand side, while the average of the 30 runs for each generation value is on the right-hand side. From Fig. 14, it can be noted that the Standard GA has initially outperformed both the Standard DE and MJPN-DE with a very quick convergence towards better solutions; however, after a short time (around 50 generations) it has been stuck on local optima. In contrast, the proposed MJPN-DE has steadily converged toward the optimal solution. In Fig. 15, the proposed MJPN-DE shows more significant progress than both GA and DE algorithms in terms of the convergence speed toward the optimal solution. Such performance of MJPN-DE approves its suitability for solving PFD with large dimensions. Although Standard DE has a

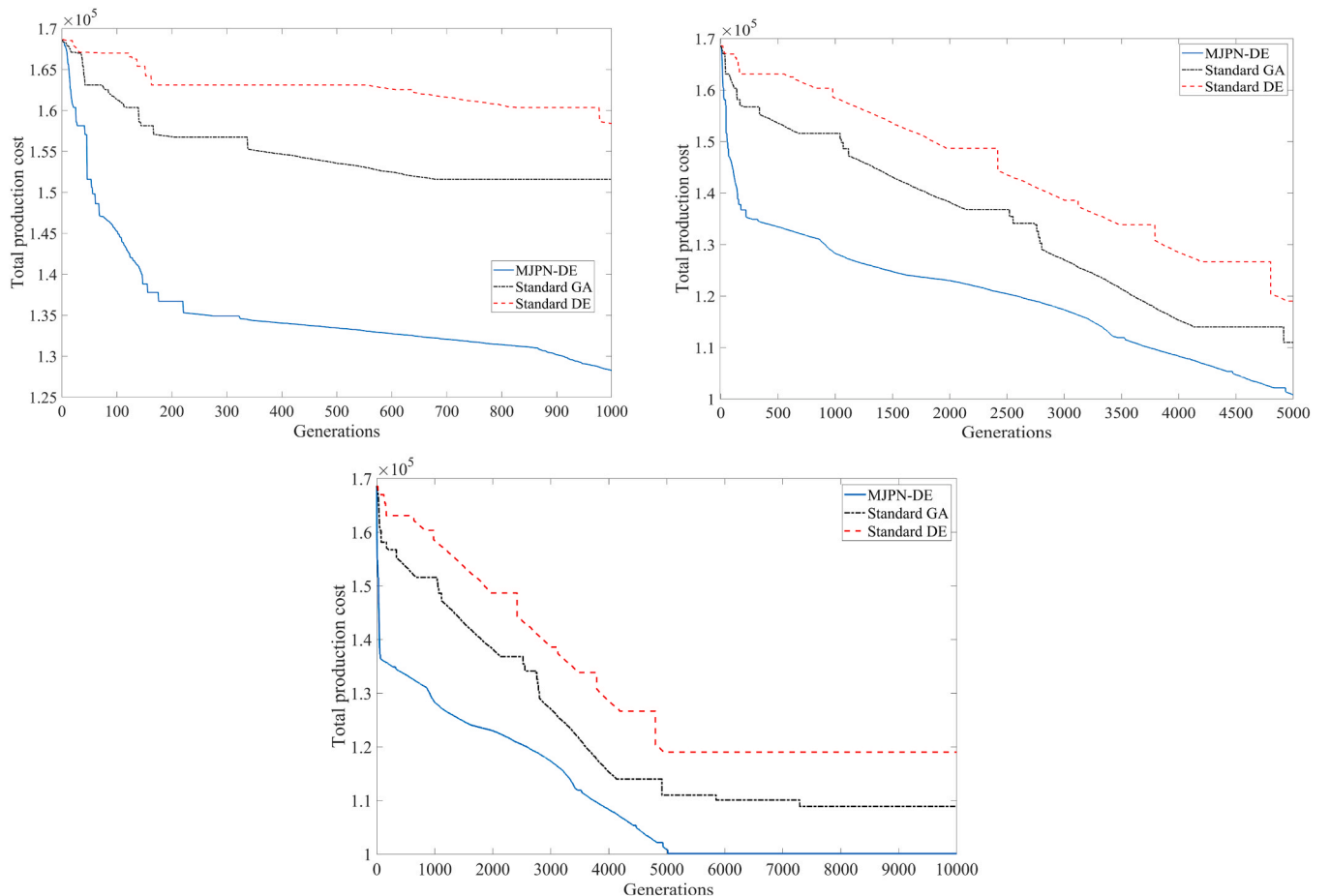


Fig. 15. Convergence plot for average runs with 1000, 5000 and 10000 generations of standard GA, standard DE and MJPN-DE – Case study II.

Table 14

RPD of the MJPN-DE based on the package-based model compared with standard GA and DE.

Techniques	Case study I			Avg. of 30 runs			Case study II			Avg. of 30 runs		
	Best run			100	500	1000	Best run			1000	5000	10000
Generations	100	500	1000	100	500	1000	1000	5000	10000	1000	5000	10000
MJPN-DE vs. Standard DE	34.96%	79.83%	78.48%	23.52%	83.29%	83.21%	19.02%	15.29%	15.49%	16.79%	17.01%	17.26%
MJPN-DE vs. Standard GA	8.88%	77.98%	77.98%	21.38%	67.81%	69.14%	15.38%	9.17%	8.17%	12.56%	10.46%	9.93%

Table 15

Average computational time (in seconds) of 30 runs of the MJPN-DE, standard GA and DE.

Generations	Case study I			Case study II		
	100	500	1000	1000	5000	10000
MJPN-DE	438	2197	4393	5529	27659	55295
Standard GA	398	1981	3987	5203	26029	52042
Standard DE	482	2410	4827	6002	30022	60026

reasonable convergence rate compared with Standard GA, it has not improved for a long time and could not achieve a better solution. Finally, the obtained results confirm that the MJPN-DE has an outstanding performance compared with both standard GA and DE algorithms.

Table 14 compares the RPD values of the MJPN-DE algorithm to the standard versions of GA and DE (in terms of best and average runs) with 100, 500, and 1000 generations (for case study I) and 1000, 5000, and 10000 generations (for case study II). Based on the RPD values from case study I, it can be shown that the MJPN-DE outperformed the Standard GA by roughly 9% in the early generations; however, because of the lack of diversity in the Standard GA's population, it was quickly trapped in local optima. The MJPN-DE, on the other hand, has continued to converge towards the optimal solution due to the improved evolving operators that enable the DE to avoid putative local optimum solutions. In case study II, the proposed MJPN-DE shows a steady speed of convergence towards optimal solutions and outperformed standard DE and GA with RPD values equal 16.90% and 11.51% (over 30 runs with 1000, 5000 and 10000 generations), respectively while solving instances with large dimensions.

To give an indication of the computational time (in seconds) of the proposed MJPN-DE algorithm, the average computational time of running MJPN-DE, standard GA and DE algorithms for 30 runs with the number of generations in each run equals 100, 500 and 1000 for case study I, and 1000, 5000, and 10000 for case study II are shown in Table 15.

From Table 15, although the average computational time of the three comparative algorithms is very close, MJPN-DE confirms the effectiveness of the proposed mechanisms/elements integrated with the DE, such as the proposed heuristic method, enhanced solution representation and BMV mapping method, on enhancing the total performance of the DE as they enable the DE to achieve a better quality of solutions in shorter computational time compared with the Standard DE.

6. Conclusion and future work

This research introduces a reactive framework for minimizing the total product cost of a PFD by integrating a classification tool with the popular DE evolutionary optimization method. In the proposed framework, the MJPN is utilized to define the number of platforms and the number of components to be assembled or disassembled to/from the assembly line of each product variant. The resultant data from the MJPN is then used as input to the proposed DE to start its evolutionary search. After the initial population is generated, a proposed package-based model is used to allow DE to begin its search with near-optimal solutions rather than randomly generated ones. The introduced package-based model distributes the components into integral parts (packages). The new model defines a newly added layer, namely package, between the product platforms and the separate components in the PFD. This layer seeks to reduce component purchase and assembly/disassembly

costs by combining components that are frequently used in the manufacturing of a certain product into a single integral part (package). In the comparison, two heuristics based on different models were used to solve a real-world case study of a kettle product family, and the results were compared to the proposed model's results to assess its efficacy.

The results show several advantages of the proposed model-based heuristic when compared to previous heuristics. First is *cost reduction* as the proposed model can achieve a remarkable 57.5% reduction in component purchasing costs and an average 41.35% reduction in setup (assembly/disassembly) processes. Second is *enhanced layouts* as the package-based model introduced in this research leads to improved layout designs, increasing postponement effectiveness by 41.51% and 74.11% compared to alternative models. This enhanced layout can result in more efficient production processes and reduced lead times. Final is *improved optimization* as the performance of the proposed MJPN-DE surpasses that of standard versions of DE and GA by wide margins, with a 63.34% and 38.52% improvement in the small case study and a 16.90% and 11.51% improvement in the large case study, respectively. This demonstrates the robustness and effectiveness of the proposed framework in solving PFD problems.

Drawbacks of the proposed framework can be summarised in *complexity* and *computational resources*. Firstly, the integration of the MJPN, DE, and package-based models may introduce added *complexity* to the PFD process. This complexity could require additional training and expertise for users to fully harness the framework's capabilities. Secondly, the computational demands of the proposed framework, especially when dealing with large-scale PFD problems, may require access to substantial computing resources. This could be a limitation for organizations with limited computational infrastructure. Finally, scalability of this proposed model has not been thoroughly investigated by solving other variants of PFD problems such as scalable-based PFD.

In the future and to address the abovementioned limitations, the effect of an enhanced version of the proposed MJPN-DE and package-based model will be further tested on more difficult PFD problems with more production factors, such as supply chain, customer satisfaction, and feedback, being considered rather than only the design phase. Considering these factors, the proposed MJPN-DE can also be implemented to solve PFD as a *multiple-objective optimization* problem in dynamic environments. Furthermore, the proposed algorithm can be extended to solve manufacturing systems with *scalable-based* PFD to test the performance of the population-based algorithms while tackling such problems.

CRediT authorship contribution statement

Ismail M. Ali: Conceptualization, Data curation, Formal analysis; Methodology, Software Roles/Writing – original draft. **Sumana Biswas:** Conceptualization, Investigation, Writing – review & editing. **Hasan Hüseyin Turan:** Investigation, Validation, Writing – review & editing. **Ripon K Chakraborty:** Formal analysis, Validation, Writing – review & editing. **Sondoss Elsayah:** Project administration; Resources; Validation, Writing – review & editing. **Michael J Ryan:** Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial

interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

I have shared a link to the data of case study 2 in the submitted file.

References

- [1] M. Bortolini, F.G. Galizia, C. Mora, Reconfigurable manufacturing systems: literature review and research trend, *J. Manuf. Syst.* 49 (2018) 93–106.
- [2] S. Borgo, et al., Knowledge-based adaptive agents for manufacturing domains, *Eng. Comput.* 35 (3) (2019) 755–779.
- [3] Huang, J. and X.J.E.O. Gu, *Distributed assembly permutation flow-shop scheduling problem with sequence-dependent set-up times using a novel biogeography-based optimization algorithm*. 2021: p. 1–21.
- [4] S. Berry, A. Pakes, The pure characteristics demand model, *Int. Econ. Rev.* 48 (4) (2007) 1193–1225.
- [5] F.S. Fogliatto, G.J.C. da Silva, D. Borenstein, The mass customization decade: An updated review of the literature, *Int. J. Prod. Econ.* 138 (1) (2012) 14–25.
- [6] I. Ullah, R. Narain, The Impact of Customer Relationship Management and Organizational Culture on Mass Customization Capability and Firm Performance, *International Journal of Customer Relationship Marketing and Management (IJCRM)* 11 (3) (2020) 60–81. <http://doi.org/10.4018/IJCRM.2020070104>.
- [7] F.G. Galizia, et al., Product platforms design, selection and customisation in high-variety manufacturing, *Int. J. Prod. Res.* 58 (3) (2020) 893–911.
- [8] M.H. Meyer, A.P. Lehnerd, The Power of Product Platforms, Simon and Schuster, 1997.
- [9] S.K. Moon, K.J. Park, T.W. Simpson, Platform design variable identification for a product family using multi-objective particle swarm optimization, *Res. Eng. Des.* 25 (2) (2014) 95–108.
- [10] F. Alizon, S.B. Shooter, T.W. Simpson, Henry Ford and the Model T: lessons for product platforming and mass customization, *Des. Stud.* 30 (5) (2009) 588–605.
- [11] M. Hanafy, H. ElMaraghy, Modular product platform configuration and co-planning of assembly lines using assembly and disassembly, *J. Manuf. Syst.* 42 (2017) 289–305.
- [12] D. Ben-Arieh, T. Easton, A. Choubey, Solving the multiple platforms configuration problem, *Int. J. Prod. Res.* 47 (7) (2009) 1969–1988.
- [13] M. Hanafy, H. ElMaraghy, Developing assembly line layout for delayed product differentiation using phylogenetic networks, *Int. J. Prod. Res.* 53 (9) (2015) 2633–2651.
- [14] H.-J. Bandelt, P. Forster, A. Röhl, Median-joining networks for inferring intraspecific phylogenies, *Mol. Biol. Evol.* 16 (1) (1999) 37–48.
- [15] T. AlGeddawy, H. ElMaraghy, Reactive design methodology for product family platforms, modularity and parts integration, *CIRP J. Manuf. Sci. Technol.* 6 (1) (2013) 34–43.
- [16] Tang, Q., et al., *An improved multi-objective multifactorial evolutionary algorithm for assembly line balancing problem considering regular production and preventive maintenance scenarios*. 2022. 68: p. 101021.
- [17] M.A. Yazdani, A. Khezri, L. Benyoucef, Process and production planning for sustainable reconfigurable manufacturing systems (SRMSs): multi-objective exact and heuristic-based approaches, *Int. J. Adv. Manuf. Technol.* (2022).
- [18] C.A. Moreira, et al., A multi-criteria h-adaptive finite-element framework for industrial part-scale thermal analysis in additive manufacturing processes, *Eng. Comput.* 38 (6) (2022) 4791–4813.
- [19] J.H. Jaramillo, J. Bhadury, R. Batta, On the use of genetic algorithms to solve location problems, *Comput. Oper. Res.* 29 (6) (2002) 761–779.
- [20] E. Erel, S.C. Sarin, A survey of the assembly line balancing procedures, *Prod. Plan. Control* 9 (5) (1998) 414–434.
- [21] E. Olivares-Benitez, J.L. Gonzalez-Velarde, A metaheuristic approach for selecting a common platform for modular products based on product performance and manufacturing cost, *J. Intell. Manuf.* 19 (5) (2008) 599–610.
- [22] M. Moussa, H. ElMaraghy, A genetic algorithm-based model for product platform design for hybrid manufacturing, *Procedia CIRP* 93 (2020) 389–394.
- [23] M. Busogi, et al., Sequence based optimization of manufacturing complexity in a mixed model assembly line, *IEEE Access* 7 (2019) 22096–22106.
- [24] M.J. Rosenblatt, R.C. Carlson, Designing a production line to maximize profit, *IEE Trans.* 17 (2) (1985) 117–122.
- [25] J.F. Kottas, H.-S. Lau, A stochastic line balancing procedure, *Int. J. Prod. Res.* 19 (2) (1981) 177–193.
- [26] I.M. Ali, D. Essam, K. Kasmarik, Novel binary differential evolution algorithm for knapsack problems, *Inf. Sci.* 542 (2020) 177–194.
- [27] L. Gauss, D.P. Lacerda, P.A. Cauchick Miguel, Module-based product family design: systematic literature review and meta-synthesis, *J. Intell. Manuf.* 32 (1) (2021) 265–312.
- [28] Jalilvand, A., S. Khanmohammadi, and F. Shabaninia. *Scheduling of sequence-dependent jobs on parallel multiprocessor systems using a branch and bound-based Petri net*. In Proceedings of the IEEE Symposium on Emerging Technologies, 2005. 2005. IEEE.
- [29] G. Gigerenzer, W. Gaissmaier, Heuristic decision making, *Annu. Rev. Psychol.* 62 (2011) 451–482.
- [30] C. Blum, A. Roli, Metaheuristics in combinatorial optimization: Overview and conceptual comparison, *ACM Comput. Surv. (CSUR)* 35 (3) (2003) 268–308.
- [31] D. Gupta, S. Benjaafar, Make-to-order, make-to-stock, or delay product differentiation? A common framework for modeling and analysis, *IEE Trans.* 36 (6) (2004) 529–546.
- [32] A. Jose, M. Tollenaere, Modular and platform methods for product family design: literature analysis, *J. Intell. Manuf.* 16 (3) (2005) 371–390.
- [33] M. Muffatto, Introducing a platform strategy in product development, *Int. J. Prod. Econ.* 60 (1999) 145–153.
- [34] C. Gurău, A. Ranchhod, Measuring customer satisfaction: a platform for calculating, predicting and increasing customer profitability, *J. Target. Meas. Anal. Mark.* 10 (3) (2002) 203–219.
- [35] E.S. Suh, et al., Flexible platform component design under uncertainty, *J. Intell. Manuf.* 18 (1) (2007) 115–126.
- [36] K. Fujita, H. Yoshida, Product variety optimization: simultaneous optimization of module combination and module attributes. *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, American Society of Mechanical Engineers, 2001.
- [37] Allada, V. and J. Lan. *New modules launch planning for evolving modular product families*. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*. 2002.
- [38] A.J. Lambert, Exact methods in disassembly sequencing as a benchmark for heuristic algorithms. *Environmentally Conscious Manufacturing IV*, International Society for Optics and Photonics, 2004.
- [39] R.B. Stone, K.L. Wood, R.H. Crawford, A heuristic method for identifying modules for product architectures, *Des. Stud.* 21 (1) (2000) 5–31.
- [40] M. Kokkolaras, et al., Extension of the target cascading formulation to the design of product families, *Struct. Multidiscip. Optim.* 24 (4) (2002) 293–301.
- [41] L. Gauss, D.P. Lacerda, M.A. Sellitto, Module-based machinery design: a method to support the design of modular machine families for reconfigurable manufacturing systems, *Int. J. Adv. Manuf. Technol.* 102 (9–12) (2019) 3911–3936.
- [42] T.-L. Yu, A.A. Yassine, D.E. Goldberg, An information theoretic method for developing modular architectures using genetic algorithms, *Res. Eng. Des.* 18 (2) (2007) 91–109.
- [43] A.J. Rojas Arciniegas, H.M. Kim, Optimal component sharing in a product family by simultaneous consideration of minimum description length and impact metric, *Eng. Optim.* 43 (2) (2011) 175–192.
- [44] Fujita, K., H. Sakaguchi, and S. Akagi. *Product variety deployment and its optimization under modular architecture and module commonalization*. In Proceedings of the 1999 ASME design engineering technical conferences. 1999. Citeseer.
- [45] T.W. Simpson, B.S. D'souza, Assessing variable levels of platform commonality within a product family using a multiobjective genetic algorithm, *Concurr. Eng.* 12 (2) (2004) 119–129.
- [46] K. Wansaeue, et al., Multiobjective meta-heuristic with iterative parameter distribution estimation for aeroelastic design of an aircraft wing, *Eng. Comput.* 38 (1) (2022) 695–713.
- [47] D'Souza, B.S. and T.W. Simpson. *A genetic algorithm based method for product family design optimization*. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*. 2002.
- [48] H. Lee, et al., An analytic network process approach to measuring design change impacts in modular products, *J. Eng. Des.* 21 (1) (2010) 75–91.
- [49] A. Beauville dit Eynaud, et al., Framework for the design and evaluation of a reconfigurable production system based on movable robot integration, *Int. J. Adv. Manuf. Technol.* 118 (7) (2022) 2373–2389.
- [50] A. Kumar, L. Pattanaik, R. Agrawal, Multi-objective Scheduling Model for Reconfigurable Assembly Systems. *Innovations in Soft Computing and Information Technology*, Springer, 2019, pp. 209–217.
- [51] L. Borba, M. Ritt, C. Miralles, Exact and heuristic methods for solving the robotic assembly line balancing problem, *Eur. J. Oper. Res.* 270 (1) (2018) 146–156.
- [52] Ö. Hazır, A. Dolgui, Assembly line balancing under uncertainty: Robust optimization models and exact solution method, *Comput. Ind. Eng.* 65 (2) (2013) 261–267.
- [53] P.M. Vilarinho, A.S. Simaria, A two-stage heuristic method for balancing mixed-model assembly lines with parallel workstations, *Int. J. Prod. Res.* 40 (6) (2002) 1405–1420.
- [54] M.C.O. Moreira, A.M. Costa, Hybrid heuristics for planning job rotation schedules in assembly lines with heterogeneous workers, *Int. J. Prod. Econ.* 141 (2) (2013) 552–560.
- [55] T. AlGeddawy, H. ElMaraghy, Assembly systems layout design model for delayed products differentiation, *Int. J. Prod. Res.* 48 (18) (2010) 5281–5305.
- [56] Hossain, M.S., et al., *Sustainable modular product architecture design by Bi-level leader-follower joint optimization with switching-based meta-heuristic algorithm*. 2021. 306: p. 127108.
- [57] D.H. Huson, C. Scornavacca, A survey of combinatorial methods for phylogenetic networks, *Genome Biol. Evol.* 3 (2011) 23–35.
- [58] I.J. Kitching, et al., *Cladistics: The Theory and Practice of Parsimony Analysis*, Oxford University Press, USA, 1998.
- [59] H. ElMaraghy, T. AlGeddawy, A. Azab, Modelling evolution in manufacturing: A biological analogy, *CIRP Ann.* 57 (1) (2008) 467–472.
- [60] Y. Zhang, X. Hu, C. Wu, Improved imperialist competitive algorithms for rebalancing multi-objective two-sided assembly lines with space and resource constraints, *Int. J. Prod. Res.* 58 (12) (2020) 3589–3617.
- [61] H. Yilmaz, M. Yilmaz, A mathematical model and tabu search algorithm for multi-manned assembly line balancing problems with assignment restrictions, *Eng. Optim.* 52 (5) (2020) 856–874.
- [62] A.C. Nearchou, Maximizing production rate and workload smoothing in assembly lines using particle swarm optimization, *Int. J. Prod. Econ.* 129 (2) (2011) 242–250.

- [63] A. Charles, G. Parks, Application of Differential Evolution algorithms to multi-objective optimization problems in mixed-oxide fuel assembly design, *Ann. Nucl. Energy* 127 (2019) 165–177.
- [64] R. Storn, K. Price, Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces, *J. Glob. Optim.* 11 (4) (1997) 341–359.
- [65] V. Plagianakos, D. Tasoulis, M. Vrahatis, A review of major application areas of differential evolution. *Advances in Differential Evolution*, Springer, 2008, pp. 197–238.
- [66] H. Zhang, et al., Differential evolution-assisted salp swarm algorithm with chaotic structure for real-world problems, *Eng. Comput.* (2022).
- [67] P.B. Myszkowski, et al., Hybrid differential evolution and greedy algorithm (DEGR) for solving multi-skill resource-constrained project scheduling problem, *Appl. Soft Comput.* 62 (2018) 1–14.
- [68] I.M. Ali, et al., A differential evolution algorithm for solving resource constrained project scheduling problems. *Artificial Life and Computational Intelligence*, Springer International Publishing, Cham, 2016.
- [69] I.M. Ali, D. Essam, K. Kasmarik, A novel design of differential evolution for solving discrete traveling salesman problems, *Swarm Evolut. Comput.* 52 (2020), 100607.
- [70] Ali, I.M., D. Essam, and K. Kasmarik. *Differential Evolution Algorithm for Multiple Inter-dependent Components Traveling Thief Problem*. in *2020 IEEE Congress on Evolutionary Computation (CEC)*. 2020.
- [71] M. Bortolini, et al., Design, engineering and testing of an innovative adaptive automation assembly system, *Assem. Autom.* 40 (3) (2020) 531–540.
- [72] K. Miettinen, M.M. Mäkelä, J. Toivanen, Numerical comparison of some penalty-based constraint handling techniques in genetic algorithms, *J. Glob. Optim.* 27 (4) (2003) 427–446.
- [73] A. Chandra, S. Chattopadhyay, Role of mutation strategies of differential evolution algorithm in designing hardware efficient multiplier-less low-pass FIR filter, *J. Multimed.* 7 (5) (2012).
- [74] I.M. Ali, D. Essam, K. Kasmarik, A novel differential evolution mapping technique for generic combinatorial optimization problems, *Appl. Soft Comput.* 80 (2019) 297–309.
- [75] I. Chivers, J. Sleightholme, An introduction to algorithms and the big O notation. *Introduction to Programming with Fortran: With Coverage of Fortran 90, 95, 2003, 2008 and 77*, Springer International Publishing, Cham, 2015, pp. 359–364.
- [76] Ruiz, R., C. Maroto, and J.J.O. Alcaraz, *Two new robust genetic algorithms for the flowshop scheduling problem*. 2006. 34(5): p. 461–476.
- [77] D. Whitley, N.-W. Yoo, Modeling simple genetic algorithms for permutation problems. *Foundations of Genetic Algorithms*, Elsevier, 1995, pp. 163–184.